Comparative Evaluation of Machine Learning Classifiers for Extreme Rainfall at the Kathmandu Airport Grid Cell, Nepal

Bishow Gurung
College ID: 260005 | Coventry University ID: 17624457
MSc Data Science and Computational Intelligence
Softwarica College of IT & E-Commerce, Kathmandu, Nepal
Email: 260005@softwarica.edu.np | Module: STW7072CEM Machine Learning

***Abstract*—Severe monsoon downpours in the Kathmandu Valley regularly cause destructive flash floods, yet identifying these rare storms from gridded weather records remains challenging because dry and moderate days vastly outnumber extreme rainfall days. Guided by three consolidated research objectives, this study develops a leak-free daily weather dataset spanning 30 years (1995–2024, 10,958 raw days and 10,951 complete records after lag creation) for the Tribhuvan International Airport grid cell. Using a strict chronological split, historical records from 1995 to 2018 (8,759 days, 439 extreme days above 13.74 mm/day) are used exclusively for model training, while records from 2019 to 2024 (2,192 days, 164 extreme days) serve as an unseen holdout test set. First, Principal Component Analysis (PCA) compresses 22 weather predictors into 7 components retaining 91.46% of data variation, Fast Independent Component Analysis (FastICA) isolates sharp storm bursts, and K-Means clustering identifies three seasonal regimes in which a single wet monsoon cluster captures 94.99% (417 of 439) of training storms. Second, five supervised classifiers are compared: Logistic Regression, Decision Tree (CART), Radial Basis Function Support Vector Machine (RBF-SVM), Random Forest, and k-Nearest Neighbours (k-NN). On the 2019–2024 test set, Random Forest delivers the best overall balance between storm detection and false-alarm control (F1-score of 0.4385, Precision-Recall AUC of 0.5517, ROC-AUC of 0.9349, and 90.24% recall), while class-weighted Logistic Regression achieves the highest storm sensitivity (96.34% recall, catching 158 of 164 storms). In contrast, unweighted k-NN reaches 93.16% overall accuracy but misses 71.34% of extreme storms (28.66% recall). Finally, lowering the Random Forest warning threshold to 0.30 based on training cross-validation under a 15-to-1 missed-flood penalty raises test recall to 98.17% (161 of 164 storms detected) and successfully flags the catastrophic 24–28 September 2024 Kathmandu flood both on the same day and 24 hours in advance.**

***Index Terms*—Cost-sensitive threshold tuning, extreme rainfall classification, imbalanced weather data, Kathmandu Valley, machine learning comparison, monsoon flood warning.**

I. Introduction

A. Background and Motivation

Heavy monsoon downpours and late-season low-pressure storms frequently trigger flash flooding and infrastructure damage across central Nepal [1, 2, 3]. Surrounded by mountain ridges rising above 2,500 metres, the bowl-shaped Kathmandu Valley (1,300–1,400 m elevation) channels runoff from the Bagmati and Bishnumati rivers through a single narrow outlet at the Chobhar gorge [4, 5]. Rapid urban growth has replaced open floodplains with concrete, meaning that daily rainfall above 14 to 25 mm/day quickly overwhelms drainage networks [5, 6, 7, 8]. During 24–28 September 2024, five consecutive days of extreme rain peaking at 93.26 mm/day at the Tribhuvan International Airport grid cell halted flights, flooded riverside settlements, and claimed over 240 lives, underscoring the urgent need for reliable statistical flood warning tools [6, 7].

While physics-based weather models capture regional circulation [9, 10, 11], their coarse grid spacing smooths out local valley cloudbursts and requires heavy computing resources [3, 12]. Machine learning classifiers offer a practical alternative by mapping daily temperature, humidity, pressure, solar radiation, and past rainfall to storm probabilities [1, 2, 12]. However, building a trustworthy classifier requires overcoming three challenges: (1) extreme rain above the 95th percentile (13.74 mm/day) occurs on only 5.01% of training days, causing standard models to favour the dry majority class [2, 3]; (2) multi-day monsoon persistence causes data leakage under random splits [1, 13]; and (3) missing a damaging flood is far costlier than issuing a precautionary standby alert [3, 6].

B. Research Aim and Objectives

The primary aim of this study is to compare supervised machine learning classifiers and unsupervised weather pattern analysis techniques for identifying extreme daily rainfall at the Kathmandu Airport grid cell, supporting reliable and cost-sensitive early flood warning. To achieve this aim, the study is structured around three consolidated Research Objectives (RO1–RO3):

1) *RO1 (Data Preparation and Unsupervised Meteorological Pattern Analysis)*

To develop a reproducible and leakage-free meteorological dataset for extreme rainfall classification at the Kathmandu Airport grid cell using NASA POWER/MERRA-2 historical observations (1995–2024), incorporating physically meaningful feature engineering, chronological data partitioning, and unsupervised techniques (PCA, FastICA, and K-Means clustering) to investigate underlying meteorological patterns and rainfall regimes.

2) *RO2 (Comparative Evaluation of Machine Learning Algorithms)*

To implement and critically compare five supervised machine learning classifiers—Logistic Regression, Decision Tree (CART), Radial Basis Function Support Vector Machine (RBF-SVM), Random Forest, and k-Nearest Neighbours (k-NN)—evaluating their effectiveness in identifying extreme rainfall events under severe class imbalance in terms of predictive performance, interpretability, probability calibration, and computational efficiency.

3) *RO3 (Operational Evaluation and Cost-Sensitive Extreme Rainfall Prediction)*

To evaluate the operational suitability and reliability of the developed classification framework through cost-sensitive threshold optimisation, strict 24-hour lead-time forecasting, wet-

day extreme rainfall classification, and retrospective validation against the September 2024 Kathmandu flood event.

II. Literature Review

A. Himalayan Extreme Rainfall Trends

Multi-decade station studies confirm that extreme monsoon rain in the Kathmandu Valley is intensifying even as total rainy days fluctuate [4, 5, 8]. Analysing station observations across the valley, Chaulagain *et al.* [4] and Khanal *et al.* [5] linked rising 95th-percentile downpours to large-scale moisture transport from the Bay of Bengal and rapid urban surface heating. Phuyal [8] further showed that heavy downpours are strongly concentrated between June and September along the valley floor and surrounding hillslopes. Across the wider Himalayan region, Wani *et al.* [9], Pujara and Paudel [10], and Bashyal *et al.* [11] demonstrated that traditional linear time-series models fail to capture sudden mountain cloudbursts, whereas complex deep neural networks require large computing budgets and dense local sensor networks [11, 12], motivating practical machine learning classifiers built on continuous satellite and reanalysis archives [13].

B. Limitations of Continuous Rainfall Regression

A recurring flaw in recent rainfall modelling is framing extreme storm detection as a continuous regression problem. On a 10-year Kathmandu city weather dataset (2015–2025), Poudyal *et al.* [1] compared Random Forest, Support Vector Regression, k-NN, and deep recurrent models using root-mean-square error and mean absolute error. Because symmetric regression losses penalise errors across all days equally, models pull predictions toward the dry average and systematically underestimate dangerous flood peaks [1, 3]. Similarly, Nurmayati [2] evaluated Random Forest, SVM, and k-NN on 52,703 ten-minute rainfall records in Indonesia; because over 90% of intervals were dry, predicting near-zero rain everywhere produced deceptively low average error but negative explanatory skill ($R^2 < 0$), while random 10-fold cross-validation leaked temporally adjacent storm observations across folds. Even state-of-the-art global weather models such as Pangu-Weather [15], GraphCast [16], GenCast [17], WeatherBench 2 [18], and Aurora [19] smooth out local upper-tail extremes [3], prompting Zhu *et al.* [20] and Talan *et al.* [16] to recommend local supervised classification using multi-day moisture and solar radiation indicators.

C. Unsupervised Clustering and Imbalanced Classification

Recent hydrometeorological studies show that pairing PCA with K-Means clustering helps compress correlated humidity and temperature variables into clear seasonal weather regimes prior to supervised modelling [16, 17], while FastICA isolates sharp, non-normal storm bursts from background seasonal cycles. In supervised flood and rainfall classification, recent benchmarks by Wu *et al.* [18], Liu *et al.* [19], and Toufique *et al.* [20] demonstrate that class-weighted models and bagged tree ensembles handle severe class skew far more reliably than unweighted neighbour search when evaluated with chronological splits, Precision-Recall curves, and calibrated decision thresholds [2, 3].

D. Identified Research Gaps Mapping to RO1–RO3

Reviewing the 2022–2026 literature reveals three clear gaps addressed by this study. First, regarding **RO1**, no Kathmandu study has combined a 30-year leak-free reanalysis pipeline with PCA, FastICA, and K-Means regime analysis to quantify how strongly extreme rain concentrates inside a specific monsoon regime. Second, regarding **RO2**, local studies have relied on bulk regression errors or unweighted k-NN without comparing linear, tree, kernel, ensemble, and instance-based classifiers on minority-class storm recall, probability calibration, and feature reliance. Third, regarding **RO3**, prior work has not tested training-only 15-to-1 cost-sensitive thresholds, strict 24-hour advance forecasting, wet-day deluges, and retrospective warning performance on the September 2024 Kathmandu flood.

III. Problem and Dataset Description

A. Problem Definition and Significance

1) Formulation of the Extreme Rainfall Classification Problem

Predicting extreme monsoon rainfall in the steep terrain of the Kathmandu Valley is formulated in this study as a supervised binary classification problem rather than a continuous regression task. While continuous regression models attempt to estimate exact daily millimetre totals across all days of the year, they are dominated by the 95% majority of dry and moderate rain days and systematically underestimate rare, flood-producing cloudbursts. By defining extreme daily rainfall through historical 95th-percentile thresholds ($P_{95} = 13.74$ mm/day across all days and $R95p = 22.75$ mm/day across wet days), the problem is structured to estimate the calibrated probability that local atmospheric conditions at the Tribhuvan International Airport grid cell will produce a hazardous downpour.

2) Practical Significance for Flood Warning and Aviation Safety

Solving this problem holds direct operational significance for municipal disaster management and aviation operations in Kathmandu. Because surface runoff from the surrounding mountain ridges converges rapidly into the urbanised Bagmati and Bishnumati river corridors, early probability alerts of extreme rainfall provide vital lead time for internal meteorologist watch duties, municipal drainage clearance, and flood preparedness in vulnerable riverside communities.

B. Study Site and 30-Year Weather Dataset

1) Kathmandu Airport Grid Cell and Seasonality

This study analyses 30 years of daily NASA POWER and MERRA-2 observations (1 January 1995 to 31 December 2024, 10,958 raw days) [4, 13] for the Tribhuvan International Airport grid cell (27.697°N, 85.359°E; 1,269.39 m elevation). Fig. 1 shows the valley location and monthly rainfall distribution, where 88.7% of extreme rain events occur between June and September.

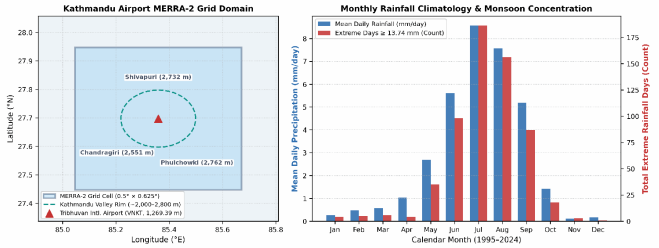


Fig. 1. Study domain showing the NASA POWER MERRA-2 grid cell centred at Tribhuvan International Airport within the Kathmandu Valley rim (left), alongside the 1995–2024 monthly mean rainfall and extreme event counts (right).

2) *Chronological Train-Test Split and Data Quality Audit*

Trimming the first 7 dry winter days of January 1995 to initialise 7-day past rainfall totals leaves 10,951 complete daily records. To prevent temporal leakage, the dataset is split chronologically at 31 December 2018 into an 80% training set (1995–2018, 8,759 days) and a 20% holdout test set (2019–2024, 2,192 days), as shown in Table I. Table II summarises the 13 raw variables. Data checks confirmed zero missing dates, while post-2012 negative calm-wind placeholder codes (-8994.0 to -5404.0) in 10m wind direction were converted into a binary calm-wind indicator.

TABLE I
CHRONOLOGICAL DATASET PARTITIONS, SAMPLE SIZES, AND EXTREME RAINFALL COUNTS

Dataset Partition	Date Range	All-Day Threshold (13.74 mm)	Wet-Day Threshold (22.75 mm)
Raw Archive	1995-01-01 to 2024-12-31	10,958 days (603 extremes)	10,958 days (212 extremes)
Lag Trim Window	1995-01-01 to 1995-01-07	7 winter days (0 extremes)	7 winter days (0 extremes)
Complete Modelling Set	1995-01-08 to 2024-12-31	10,951 days (603 extremes, 5.51%)	10,951 days (212 extremes, 1.94%)
Training Set (80%)	**1995-01-08 to 2018-12-31**	**8,759 days (439 extremes, 5.01%)**	**8,759 days (148 extremes, 1.69%)**
Holdout Test Set (20%)	**2019-01-01 to 2024-12-31**	**2,192 days (164 extremes, 7.48%)**	**2,192 days (64 extremes, 2.92%)**

TABLE II
RAW METEOROLOGICAL VARIABLES AND TRAINING SUMMARY STATISTICS (1995–2018, 8,759 DAYS)

Variable	Units	Physical Weather Role	Train Mean ± SD	Min / Max
PRECTOTCORR	mm/day	Daily total rainfall (target source)	2.59 ± 5.72	0.00 / 60.01
T2M	°C	Mean 2m air temperature	19.22 ± 4.98	4.11 / 30.11
T2M_MAX	°C	Maximum daily 2m temperature	24.72 ± 4.64	9.93 / 37.30
T2M_MIN	°C	Minimum daily 2m temperature	15.10 ± 5.08	-0.12 / 24.91
T2MDEW	°C	Dew point temperature at 2m	8.78 ± 9.35	-16.63 / 22.26
RH2M	%	Relative humidity at 2m	57.31 ± 23.06	5.43 / 96.45
QV2M	g/kg	Specific humidity (water vapour)	9.67 ± 5.38	0.97 / 19.56
PS	kPa	Surface air pressure	87.27 ± 0.40	85.92 / 88.20
WS10M	m/s	Mean 10m wind speed	2.17 ± 0.52	0.86 / 4.65
WS10M_MAX	m/s	Peak daily 10m wind gust	4.49 ± 1.02	1.72 / 9.55
WD10M*	Degrees	10m wind direction (non-calm days)	217.8 ± 43.7	1.20 / 359.9
ALLSKY_SW	MJ/m^2/d	Downward solar radiation	17.60 ± 4.69	2.21 / 30.54
TS	°C	Surface skin temperature	18.46 ± 6.43	2.60 / 31.46

*Note: Post-2012 negative calm-wind codes in WD10M are captured via a binary calm-wind indicator.

C. *Target Thresholds and Operational Classification Tasks*

1) *Extreme Rainfall Thresholds (13.74 mm and 22.75 mm)*

Two extreme rainfall thresholds are calculated strictly from the 1995–2018 training set [3, 4]: the all-day 95th percentile (13.74 mm/day, giving 439 training extremes at 5.01% and 164 test extremes at 7.48%) and the wet-day 95th percentile across days with at least 1.0 mm of rain (22.75 mm/day, giving 148 training and 64 test deluges), as plotted in Fig. 2.

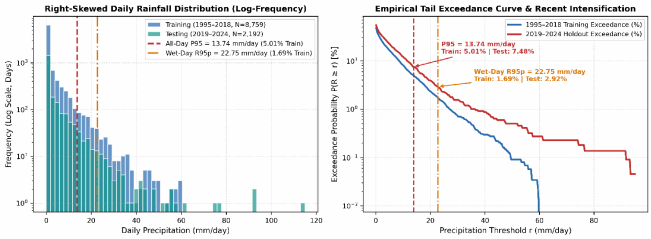


Fig. 2. Daily rainfall frequency distribution (left) and exceedance probability curves comparing the 1995–2018 training period against the 2019–2024 holdout period (right) at the 13.74 mm/day and 22.75 mm/day thresholds.

2) *Operational Classification Tasks (Tasks A, B, and C)*

To evaluate both same-day weather diagnosis and true advance warning under RO3, three classification tasks are defined:

- **Task A (Same-Day Atmospheric State Classification at 13.74 mm/day):** Uses same-day non-rainfall weather conditions (humidity, temperature, pressure, wind, and solar radiation) together with 1-to-7-day past rainfall to diagnose whether that same day experiences extreme rainfall ($\geq$ 13.74 mm/day).
- **Task B (Strict 24-Hour Advance Forecasting at 13.74 mm/day):** Shifts all 15 weather state variables back by one full day so that only observations recorded on or before yesterday are used to forecast tomorrow's extreme rainfall a full 24 hours in advance.
- **Task C (Severe Wet-Day Deluge Classification at 22.75 mm/day):** Evaluates classifier ability to detect severe wet-day downpours exceeding the 22.75 mm/day threshold.

IV. Methods and Experimental Setup

A. *Experimental Setup, Data Pre-Processing, and Feature Extraction*

1) *Leak-Free Pre-Processing and Feature Extraction Pipeline*

Fig. 3 shows the leak-free workflow and 5-fold expanding-window time-series validation design, and Table III lists the 22 engineered predictors. Inside each training fold, continuous weather predictors are standardised using z-score normalisation so that variables measured in different units share a common scale:

$$z_{i,j} = \frac{x_{i,j} - \mu_j}{\sigma_j} \tag{1}$$

where $x_{i,j}$ is the raw value of feature j on day i, and μ_j and σ_j are the mean and standard deviation calculated strictly from the historical training records.

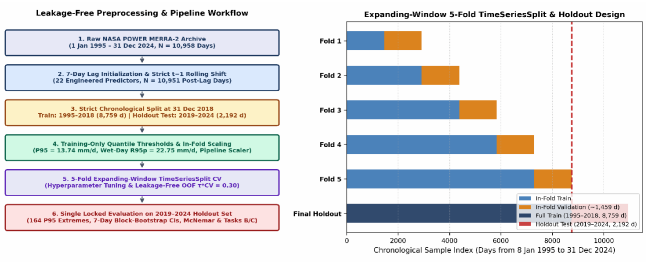


Fig. 3. Leak-free data preparation and evaluation workflow (left) and the 5-fold expanding-window time-series validation schedule applied strictly within the 1995–2018 training set (right).

TABLE III
INVENTORY OF THE 22 ENGINEERED WEATHER PREDICTORS

#	Predictor Name	Calculation Method and Units	Physical Weather Meaning
1	T2M	Daily mean 2m air temperature (°C)	Warmth governing moisture capacity
2	T2M_MAX	Daily maximum 2m temperature (°C)	Daytime surface heating peak
3	T2M_MIN	Daily minimum 2m temperature (°C)	Nighttime cloud heat trapping
4	temp_diurnal_range	Max minus min daily temperature (°C)	Cloud cooling (narrows on rainy days)
5	T2MDEW	Dew point temperature at 2m (°C)	Condensation temperature threshold
6	dew_point_depression	Air temperature minus dew point (°C)	Near-surface air saturation gap
7	RH2M	Relative humidity at 2m (%)	Near-surface moisture saturation
8	QV2M	Specific humidity at 2m (g/kg)	Water vapour fuel for storm clouds
9	PS	Surface atmospheric pressure (kPa)	Monsoon low-pressure trough depth
10	pressure_tendency	Today minus yesterday pressure (kPa)	24-hour barometric rise or fall
11	WS10M	Mean 10m wind speed (m/s)	Valley moisture transport speed
12	WS10M_MAX	Maximum daily 10m wind gust (m/s)	Storm downdraught gust strength
13	ALLSKY_SW	Downward solar radiation (MJ/m^2/d)	Sunlight blocked by storm clouds
14	TS	Surface skin temperature (°C)	Land-surface heating state
15	wind_calm_flag	Binary flag for negative wind code (0/1)	Calm or variable valley wind state
16	precip_lag1	Rainfall 1 day ago (mm/day)	Yesterday's rainfall persistence
17	precip_lag2	Rainfall 2 days ago (mm/day)	48-hour storm track memory
18	precip_lag3	Rainfall 3 days ago (mm/day)	72-hour wet spell persistence
19	precip_roll3	Total rainfall over past 3 days (mm)	Short-term soil and air wetting
20	precip_roll7	Total rainfall over past 7 days (mm)	Weekly catchment moisture build-up
21	sin_doy	Sine wave of calendar day of year	Smooth seasonal progression (spring/autumn)
22	cos_doy	Cosine wave of calendar day of year	Smooth winter-to-monsoon cycle

B. *Unsupervised Pattern Analysis and Clustering Methods (RO1)*

1) *Principal Component Analysis (PCA)*

PCA is selected to measure overlap among the 22 predictors and test whether compressing them into uncorrelated axes retains extreme rain signals [17]. It is fitted strictly on the standardised 1995–2018 training matrix (8,759 days across 22 features) without access to rainfall labels or test data. Configured in Scikit-learn with random seed 42 (fit time 0.0028 s, Table IV), the number of retained components m is selected using the cumulative explained variance ratio:

$$\mathrm{EVR}_m = \frac{\sum_{k=1}^{m} \lambda_k}{\sum_{j=1}^{22} \lambda_j} \geq 0.90 \tag{2}$$

where λ_k is the eigenvalue of component k, which retains $m = 7$ components explaining 91.46% of total training variance. PCA advances RO1 by isolating monsoon moisture saturation (Component 1, 42.29%) and solar heating (Component 2, 24.17%), but compressing 22 features into 7 components reduces downstream storm recall because global variance smoothing blurs sharp 1-day rain spikes.

2) *Fast Independent Component Analysis (FastICA)*

FastICA is selected as an exploratory diagnostic method to separate sharp, non-normal storm spikes from smooth seasonal cycles. It is applied to the standardised 8,759-day training table after 7-component whitening as an unsupervised diagnostic tool rather than a supervised classifier, configured in Scikit-learn with 7 components, a log-cosh contrast function, 500 maximum iterations, and seed 42 (fit time 0.1383 s, Table IV). This supports RO1 by separating the post-2012 calm-wind shift from sharp multi-day rain bursts, though its unitless axes are less interpretable than physical features for tree splits.

3) *K-Means Clustering*

K-Means clustering is selected to group daily valley weather states into objective seasonal regimes and measure storm concentration [17]. It is applied to the unlabelled standardised training matrix across 2 to 6 candidate clusters before comparing cluster assignments against historical storm labels, configured in Scikit-learn with 3 clusters, 10 initialisations, and seed 42 (fit time 0.0804 s, Table IV) to partition daily weather states by minimising the Within-Cluster Sum of Squares (WCSS):

$$J_{\mathrm{WCSS}} = \sum_{k=1}^{K} \sum_{z_i \in C_k} \|z_i - \mu_k\|_2^2 \tag{3}$$

where C_k is weather regime cluster k and μ_k is its cluster centre across the 22 standardised features. K-Means advances RO1 by identifying an Active Saturated Monsoon cluster that captures 94.99% (417 of 439) of training storms, though 87.35% of days in that cluster remain non-extreme, requiring supervised classifiers for precise warning.

C. *Supervised Classification Algorithms (RO2 and RO3)*

1) *Logistic Regression*

Logistic Regression is selected as an interpretable linear baseline to test weighted straight-line separation of extreme rain days [18]. It is trained on all 22 standardised predictors inside an in-fold scaling pipeline and configured in Scikit-learn with L2 penalty, the LBFGS solver, balanced class weights (19-to-1 storm weighting), and a tuned regularisation strength of $C = 0.01$ (Table V), minimising the weighted cross-entropy loss function:

$$L_{\mathrm{LR}} = -\frac{1}{N} \sum_{i=1}^{N} [w_1 y_i \ln(\hat{p}_i) + w_0 (1 - y_i) \ln(1 - \hat{p}_i)] + \frac{\|w\|_2^2}{2C} \tag{4}$$

where $\hat{p}_i$ is the predicted storm probability on day i, and $w_1 = 9.98$ and $w_0 = 0.53$ are the inverse-frequency class weights that counter the 1-to-19 imbalance. Logistic Regression achieves the highest test storm recall (96.34%, 158 of 164 storms, ROC-AUC 0.9411), but its flat boundary flags 523 humid monsoon days as false alarms (23.20% precision).

2) *Decision Tree (CART)*

Decision Tree (CART) is selected as a rule-based baseline that splits weather variables along clear step thresholds [16]. Trained on the 22 unscaled physical features so split rules remain readable in original physical units, it is configured in Scikit-learn with Gini impurity, balanced class weights, maximum depth 8, minimum split 10, and minimum leaf size 5 (Table V). On the holdout test set, CART reaches 85.98% recall and an F1-score of 0.4046, but suffers from single-feature dominance by placing 73.35% of its total decision weight on yesterday's rainfall alone.

3) *Radial Basis Function Support Vector Machine (RBF-SVM)*

RBF-SVM is selected to test whether a smooth, curved kernel boundary improves storm separation over linear and single-tree models [2]. Trained on the 22 standardised predictors inside an in-fold scaling pipeline, it is configured in Scikit-learn with an RBF kernel, penalty C of 1.0, automatic kernel scaling, balanced weights, and Platt probability calibration (Table V). RBF-SVM achieves the best probability calibration (Brier 0.0553) and 87.80% recall, but trails Random Forest in Precision-Recall AUC (0.3676) and requires the longest training time (10.45 s).

4) *Random Forest*

Random Forest is selected as a bagged tree ensemble designed to prevent single-variable overfitting and reduce false alarms [1, 18, 19]. Trained on the 22 physical predictors across 100 bootstrap trees while restricting each split to 4 randomly chosen candidate features, it is configured in Scikit-learn with 100 trees, maximum depth 8, minimum leaf size 5, balanced class weights, and seed 42 (fit time 1.29 s, Table IV and Table V). Random Forest delivers the best overall balance on the holdout

set (F1 of 0.4385, Precision-Recall AUC 0.5517, ROC-AUC 0.9349, and 90.24% recall), with only minor first-day lag delay after dry spells.

5) k-Nearest Neighbours (k-NN)

k-NN is selected as a distance-based baseline to test whether matching days to closest historical weather records works under class skew [2, 20]. Trained on the 22 standardised predictors using straight-line Euclidean distance and configured in Scikit-learn with 5 nearest neighbours and uniform voting (Table V), k-NN exposes the accuracy paradox: despite 93.16% overall accuracy, it misses 71.34% of extreme storms (28.66% recall) because non-extreme days dominate local 5-day neighbourhoods.

D. Validation and Cost-Sensitive Threshold Design

1) Time-Series Cross-Validation and Evaluation Metrics

All models are tuned via 5-fold expanding-window time-series cross-validation on 1995–2018 (Table IV and Table V). Because overall accuracy is misleading on imbalanced data, models are evaluated using Precision, Recall, the harmonic F_1-score, and the Brier probability calibration score (BS):

$$\begin{aligned} \text{Prec} &= \frac{\text{TP}}{\text{TP}+\text{FP}}, \quad \text{Rec} = \frac{\text{TP}}{\text{TP}+\text{FN}}, \\ F_1 &= \frac{2\,\text{TP}}{2\,\text{TP}+\text{FP}+\text{FN}}, \quad \text{BS} = \frac{1}{N}\sum_{i=1}^{N}(\hat{p}_i - y_i)^2 \end{aligned} \tag{5}$$

where TP, FP, and FN denote true positives, false positives, and false negatives.

2) Asymmetric 15-to-1 Disaster Cost Function

Under RO3, decision thresholds τ from 0.05 to 0.95 are evaluated using an asymmetric 15-to-1 **missed-storm cost function** that weights each missed flood 15 times more heavily than a false alarm:

$$\text{Cost}(\tau) = 15 \times \text{FN}(\tau) + 1 \times \text{FP}(\tau) \tag{6}$$

selecting the operational threshold of 0.30 strictly from out-of-fold training predictions without viewing the holdout test set.

TABLE IV
TRAINING RUNTIMES, TEST INFERENCE SPEED, AND SCALING BEHAVIOUR ACROSS ALL EIGHT ALGORITHMS

Method / Classifier	Scaling Behaviour	Train Fit Time	Test Inference (2,192 d)
PCA (22 features)	Fast matrix decomposition	0.0028 s	Not applicable
FastICA (7 components)	Fast iterative projection	0.1383 s	Not applicable
K-Means (3 clusters)	Fast centroid distance updates	0.0804 s	Not applicable
Logistic Regression	Fast linear gradient optimisation	0.0533 s	0.0011 s
Decision Tree (CART)	Fast single-tree threshold sorting	0.0576 s	0.0006 s
RBF-Kernel SVM	Slow pairwise kernel optimization	10.4518 s	0.7772 s
Random Forest (100 trees)	**Moderate parallel tree building**	**1.2909 s**	**0.0290 s**
k-NN (5 neighbours)	Instant storage; slow test distance scan	0.0024 s	0.0702 s

TABLE V
FIVE-FOLD TIME-SERIES CROSS-VALIDATION SEARCH GRIDS AND SELECTED MODEL CONFIGURATIONS

Classifier	Candidate Settings Tested	Selected Best Configuration	Train CV F1 (± SD)
Logistic Regression	Penalty C: 0.01, 0.1, 1.0, 10.0	C = 0.01, L2 penalty, balanced weights	0.3172 ± 0.0778
Decision Tree (CART)	Depth: 3, 4, 5, 6, 8; Leaf: 5, 10, 20	Max depth = 8, min leaf = 5, balanced	0.3383 ± 0.0566
RBF-Kernel SVM	Penalty C: 0.5, 1.0, 2.0; scale kernel	C = 1.0, RBF kernel, balanced, Platt	0.3476 ± 0.0514
Random Forest	**Trees: 100, 150; Depth: 4, 6, 8; Leaf: 5, 10**	**100 trees, max depth = 8, min leaf = 5**	**0.3780 ± 0.0429**
k-NN	Neighbours: 5, 9, 15, 21; uniform/distance	5 neighbours, uniform voting	0.2003 ± 0.0844

V. Results

A. Unsupervised Regime Findings (RO1)

1) Optimal Cluster Selection

Applying PCA, FastICA, and K-Means to the 1995–2018 training set (8,759 days across 22 features) reveals clear seasonal weather structure in the Kathmandu Valley. As shown in Fig. 4, testing cluster counts from 2 to 6 identifies a clear optimum at 3 clusters, where the Silhouette score reaches its global peak of 0.3119 and the Davies–Bouldin index reaches its lowest value of 1.1327. This confirms that daily Kathmandu weather naturally divides into three macro-seasonal regimes rather than an arbitrary number of groups.

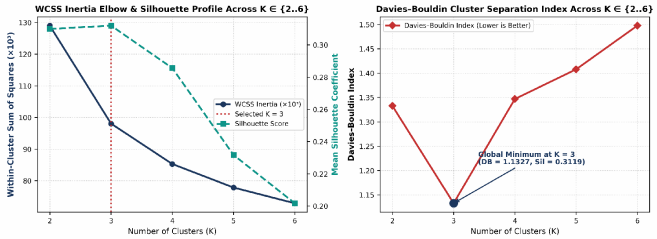


Fig. 4. Unsupervised K-Means cluster validation across 2 to 6 clusters on the 1995–2018 training set: inertia elbow and Silhouette peak at 3 clusters (left), and Davies–Bouldin minimum at 3 clusters (right).

2) PCA and FastICA Component Structure

Fig. 5 and Fig. 6 show how PCA and FastICA organise the 22 weather predictors. In Fig. 5 (left) and Fig. 6 (left), the first principal component accounts for 42.29% of total training variance and is driven by high specific humidity (+0.313), high dew point (+0.307), warm nighttime minimum temperature (+0.282), high relative humidity (+0.265), and heavy 7-day past rainfall (+0.261), paired with low surface pressure (-0.245) and narrow daily temperature range (-0.233). Physically, this first axis represents warm, saturated monsoon low-pressure conditions. The second component explains 24.17% of variance (bringing the two-axis total to 66.46%) and reflects strong 10m wind speed (+0.360), high daytime temperature (+0.357), and bright solar radiation (+0.332) typical of clear pre-monsoon days. The third component (9.15% variance) tracks 3-day and 2-day past rainfall persistence (+0.440 and +0.407). Together, the top 7 components retain 91.46% of total variance (and 9 components retain 95.11%). In Fig. 5 (right), FastICA separates the post-2012 calm-wind archive shift along its first axis while isolating sharp, heavy multi-day storm spikes along its second axis, confirming that extreme rain events stand out as distinct non-normal bursts.

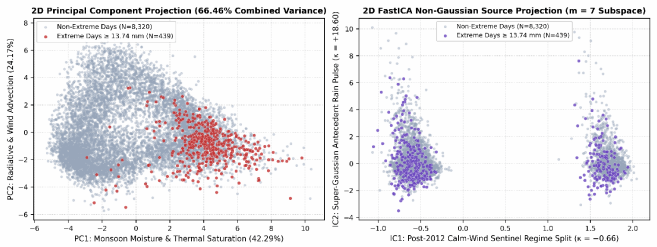


Fig. 5. Two-dimensional projections of the 22-feature training set coloured by extreme rainfall status: PCA projection capturing 66.46% of variance (left) and FastICA independent source projection (right).

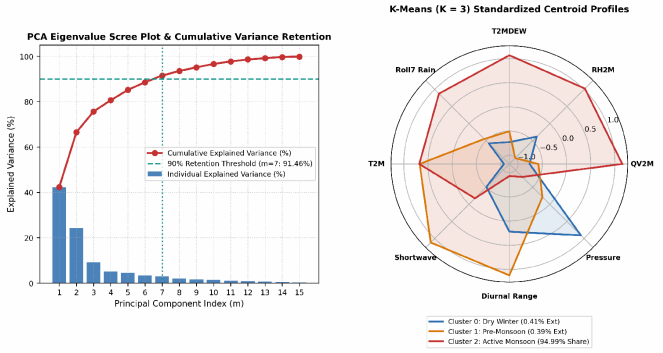


Fig. 6. PCA explained variance scree plot showing 91.46% retention at 7 components (left), paired with the K-Means (3 clusters) radar profile across eight core weather variables (right).

3) Physical Profiles of the Three Weather Regimes

Table VI and Fig. 6 (right) summarise the average weather conditions and extreme storm counts across the three K-Means clusters on the 1995–2018 training set:

- **Cluster 0 (Dry Winter and Post-Monsoon Regime):** Covers 3,395 training days (38.76%) characterised by cool air (13.90°C), low specific humidity (5.57 g/kg), high air pressure (87.63 kPa), and minimal 7-day rain (2.28 mm). Only 14 extreme rain events occur in this regime (0.41% of days, or 3.19% of all training storms).
- **Cluster 1 (Hot Pre-Monsoon Transitional Regime):** Covers 2,067 training days (23.60%) with warm temperatures (22.56°C), strong sunshine (22.82 MJ/m^2/day), wide daily temperature swings (13.02°C), and dry air (34.02% relative humidity). Only 8 extreme events fall in this cluster (0.39% of days, or 1.82% of storms).
- **Cluster 2 (Active Saturated Monsoon Regime):** Covers 3,297 training days (37.64%) featuring high water vapour content (15.85 g/kg), near-saturated relative humidity (80.92%), high dew point (18.76°C), narrow daily temperature range from thick cloud cover (6.80°C), low surface pressure (86.95 kPa), and high 7-day past rainfall (42.26 mm). Without ever seeing rainfall labels during clustering, Cluster 2 captures **nearly all training extreme events** (417 of 439 storms, or 94.99%). However, because 2,880 days inside Cluster 2 are still non-extreme monsoon days (giving a 12.65% storm rate inside the cluster), supervised classifiers are essential to separate destructive downpours from ordinary wet monsoon days.

TABLE VI
UNSUPERVISED K-MEANS (3 CLUSTERS) WEATHER REGIME PROFILES AND EXTREME STORM CONCENTRATION (1995–2018)

Weather Variable / Metric	Cluster 0: Dry Winter	Cluster 1: Pre-Monsoon	Cluster 2: Active Monsoon	All Training Days
Total Training Days (Share)	3,395 (38.76%)	2,067 (23.60%)	**3,297 (37.64%)**	8,759 (100.0%)
Specific Humidity QV2M (g/kg)	5.57	6.54	**15.85**	9.67
Relative Humidity RH2M (%)	48.56	34.02	**80.92**	57.31
Dew Point T2MDEW (°C)	1.97	4.07	**18.76**	8.78
Mean Temperature T2M (°C)	13.90	22.56	**22.60**	19.22
Daily Temp. Range DTR (°C)	10.29	13.02	**6.80**	9.62
Surface Pressure PS (kPa)	87.63	87.18	**86.95**	87.27
Solar Radiation (MJ/m^2/d)	15.20	22.82	**16.77**	17.60
7-Day Past Rainfall (mm)	2.28	5.83	**42.26**	18.17
Extreme Rain Days (Rate)	14 (0.41%)	8 (0.39%)	**417 (12.65%)**	439 (5.01%)
Share of All 439 Extremes	3.19%	1.82%	**94.99%**	100.00%

B. Supervised Classifier Comparison (RO2)

1) Holdout Test Performance Across All Five Models

Table VII presents the out-of-sample evaluation of all five supervised classifiers on the unseen 2019–2024 holdout test set (2,192 days containing 164 extreme rain events, or 7.48% positive prevalence) at the standard 0.50 probability cutoff. Fig. 7, Fig. 8, and Fig. 9 show the corresponding ROC curves, Precision-Recall curves, and five-model confusion matrices.

TABLE VII
HOLDOUT TEST PERFORMANCE ACROSS ALL FIVE SUPERVISED CLASSIFIERS (2019–2024, 2,192 DAYS, 164 EXTREMES, CUTOFF 0.50)

Evaluation Metric	Logistic Reg.	Decision Tree	RBF-SVM	Random Forest	k-NN (k=5)
Overall Accuracy (%)	75.87	81.07	79.06	82.71	**93.16**
Precision (%)	23.20	26.45	24.70	28.96	**58.75**
Recall / Storm Detection (%)	**96.34**	85.98	87.80	**90.24**	28.66
F1-Score	0.3740	0.4046	0.3855	**0.4385**	0.3852
95% Block-Bootstrap CI (F1)	[31.97, 42.63]	[34.75, 45.44]	[33.07, 43.70]	**[38.17, 49.14]**	[30.77, 44.90]
Specificity (%)	74.21	80.67	78.35	82.10	**98.37**
ROC-AUC	**0.9411**	0.8718	0.9011	**0.9349**	0.8219
Precision-Recall AUC	0.5478	0.4068	0.3676	**0.5517**	0.3547
Brier Calibration Score	0.1621	0.1418	**0.0553**	0.1154	0.0560
True Positives (Caught Storms)	**158**	141	144	**148**	47
False Negatives (Missed Storms)	**6**	23	20	**16**	117
False Positives (False Alarms)	523	392	439	363	**33**
True Negatives (Correct Dry)	1,505	1,636	1,589	1,665	**1,995**

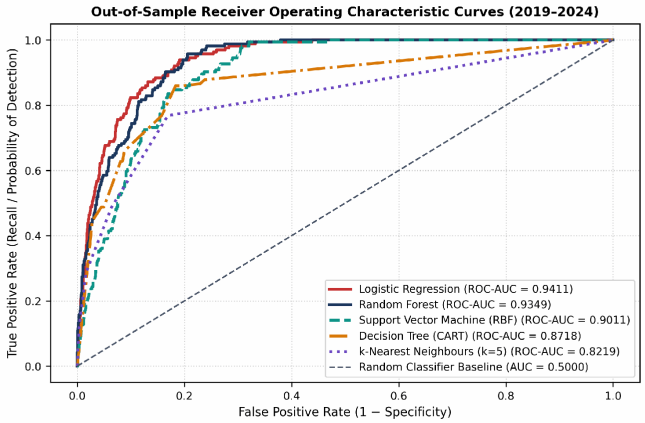


Fig. 7. Receiver Operating Characteristic (ROC) curves across all five classifiers on the 2019–2024 holdout test set (2,192 days).

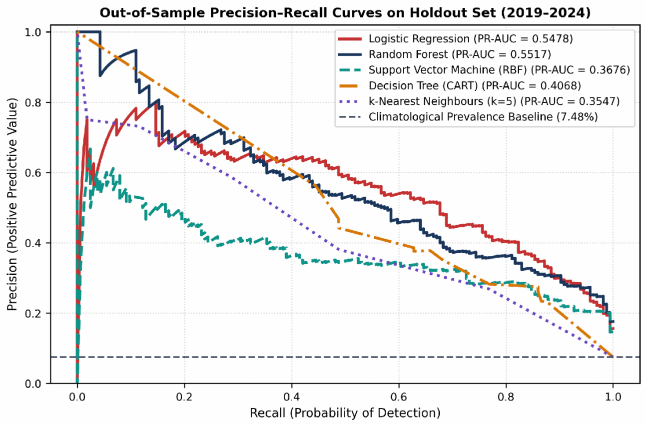


Fig. 8. Precision-Recall curves on the 2019–2024 holdout test set against the 7.48% storm prevalence baseline, showing Random Forest (0.5517) and Logistic Regression (0.5478) achieving over a seven-fold gain above random chance.

2) Random Forest: Best Overall Operational Balance

Comparing the five models in Table VII and Figs. 7–9 shows that Random Forest records the highest holdout F1-score (0.4385, with a 95% block-bootstrap confidence interval of 38.17% to 49.14%) and the highest Precision-Recall AUC (0.5517, more than 7.3 times higher than the 0.0748 no-skill baseline), alongside an ROC-AUC of 0.9349, 82.71% accuracy, and 90.24% recall (catching 148 of 164 extreme storms). Compared with a single Decision Tree (F1 of 0.4046, 23 missed storms, and 392 false alarms), combining 100 trees with ran-

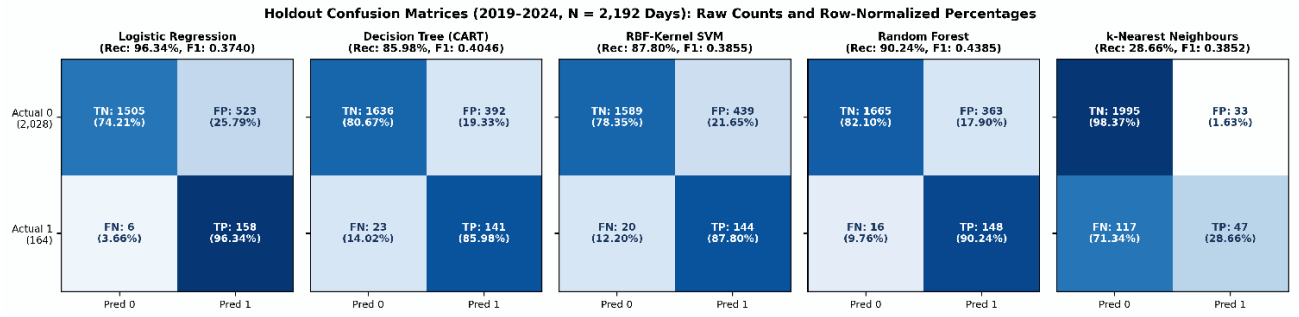


Fig. 9. Holdout confusion matrices across all five classifiers (2019–2024, 2,192 days: 2,028 non-extreme, 164 extreme) showing raw counts and row percentages.

dom 4-feature splits catches 7 additional extreme storms while eliminating 29 false alarms.

3) *Logistic Regression: Maximum Storm Sensitivity*

Class-weighted Logistic Regression achieves the highest default-cutoff recall of 96.34% (catching 158 of 164 extreme rain days and missing only 6 storms across 6 years) and the highest ROC-AUC (0.9411). However, because its straight-line boundary cannot bend around warm, humid monsoon break days, it produces 523 false alarms (160 more false alarms than Random Forest), resulting in 23.20% precision and an F1-score of 0.3740.

4) *RBF-SVM: Strong Calibration but Lower Precision-Recall Skill*

RBF-SVM detects 144 of 164 storms (87.80% recall, ROC-AUC of 0.9011) and achieves the lowest Brier calibration error (0.0553) thanks to internal Platt scaling, but its Precision-Recall AUC (0.3676) lags substantially behind Random Forest (0.5517) because of high-probability false alarms on humid days.

5) *k-Nearest Neighbours: Demonstration of the Accuracy Paradox*

Unweighted k-NN (5 neighbours) appears best if judged by overall accuracy (93.16%) or precision (58.75%), yet it misses 117 out of 164 extreme storms (28.66% recall, catching only 47 events). Because 92.52% of test days are non-extreme, a trivial model that predicts "no extreme rain" every single day achieves 92.52% accuracy while catching zero floods.

6) *Feature Importance and Physical Drivers*

Table VIII and Fig. 10 compare how Random Forest, Decision Tree (CART), Logistic Regression, and test-set permutation importance rank the top 10 weather predictors. This comparison explains why Random Forest outperforms a single Decision Tree. When a single CART tree evaluates all 22 predictors greedily at every split, it assigns 73.35% of its total decision weight to yesterday's rainfall (`precip_lag1`) while giving 0.00% weight to specific humidity (`QV2M`) and 7-day rainfall accumulation (`precip_roll7`). In contrast, because Random Forest restricts each split to 4 randomly chosen predictors, it spreads decision weight across multi-day rainfall memory (`precip_lag1` at 20.83%, `precip_roll3` at 11.55%, and `precip_roll7` at 5.24%), atmospheric moisture (`QV2M` at 10.37%, `T2MDEW` at 9.36%, `RH2M` at 5.43%, and `dew_point_depression` at 5.12%), and cloud cooling (`temp_diurnal_range` at 7.00% and `ALLSKY_SW` at 4.41%). Logistic Regression weights and permutation tests confirm the same physical mechanism: high previous-day rain and high relative humidity strongly increase storm probability, whereas bright sunshine and wide daily temperature ranges strongly indicate dry weather.

TABLE VIII
TOP-10 WEATHER PREDICTOR IMPORTANCE RANKINGS ACROSS MODELS

Rank	Predictor Variable	Random Forest	Decision Tree	Logistic Reg. Weight	Permutation Drop
1	precip_lag1 (Yesterday rain)	**0.2083**	**0.7335**	**+0.4969**	**0.0721**
2	precip_roll3 (3-day rain)	0.1155	0.0109	+0.1092	0.0105
3	QV2M (Specific humidity)	0.1037	0.0000	+0.2888	0.0034
4	T2MDEW (Dew point)	0.0936	0.0037	+0.3057	0.0012
5	temp_diurnal_range	0.0700	0.0461	-0.4886	0.0099
6	RH2M (Relative humidity)	0.0543	0.0122	+0.4679	0.0070
7	precip_roll7 (7-day rain)	0.0524	0.0000	+0.0756	0.0018
8	dew_point_depression	0.0512	0.0086	-0.3168	0.0039
9	ALLSKY_SW (Solar radiation)	0.0441	0.0322	**-0.5355**	0.0206
10	T2M_MIN (Min temperature)	0.0349	0.0097	+0.2000	0.0005

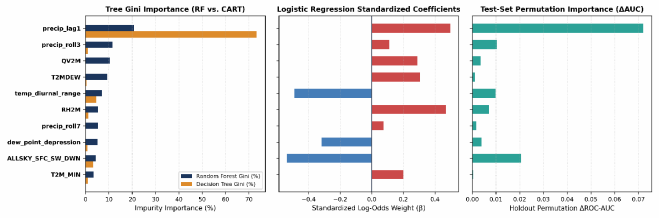


Fig. 10. Comparison of top-10 weather predictors: Random Forest versus Decision Tree Gini importance (left), Logistic Regression standardized weights (centre), and holdout test permutation importance (right).

7) *Probability Calibration and Subspace Ablation*

Fig. 11 compares the probability calibration curves and Brier scores across all five models. Platt-scaled RBF-SVM (0.0553) and unweighted k-NN (0.0560) record the lowest raw Brier scores because their predicted probabilities stay close to the 7.48% average storm rate, whereas balanced class weighting in Random Forest (0.1154), CART (0.1418), and Logistic Regression (0.1621) intentionally pushes storm probabilities upward to prevent missed floods. Table IX reports paired McNemar statistical comparisons alongside the PCA dimensionality reduction ablation test. Part A confirms that Random Forest catches 101 extreme storms missed by k-NN while k-NN catches zero storms missed by Random Forest (highly significant with $p < 10^{-30}$). Part B shows that replacing the 22 physical features with the 7 PCA components reduces Logistic Regression recall from 96.34% to 93.29% (F1 dropping from 0.3740 to 0.3562) and reduces k-NN recall from 28.66% to 24.39% (F1 dropping from 0.3852 to 0.3265), confirming that retaining all 22 physical predictors preserves vital storm precursors.

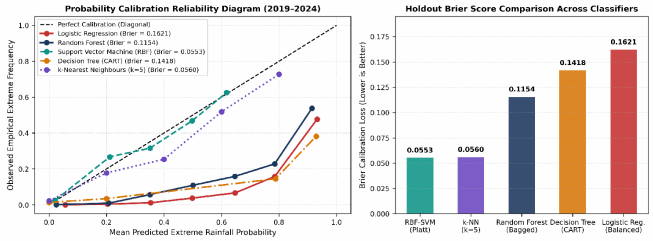


Fig. 11. Probability calibration curves across all five classifiers on the 2019–2024 holdout test set (left) and Brier score comparison showing the effect of balanced class weighting versus Platt scaling (right).

TABLE IX
PAIRED MCNEMAR SIGNIFICANCE TESTS AND 7-COMPONENT PCA VERSUS 22-FEATURE ABLATION (2019–2024 TEST SET)

Part A: Paired Model Comparison	**All Days Disagreed**	**Chi-Square (p-value)**	**Storms Disagreed**	**Storm Chi-Square (p)**
Random Forest vs. k-NN	106 vs. 335	117.88 ($< 10^{-28}$)	**101 vs. 0**	**99.01** ($< 10^{-30}$)
Logistic Regression vs. k-NN	112 vs. 491	236.96 ($< 10^{-56}$)	**111 vs. 0**	**109.01** ($< 10^{-33}$)
Random Forest vs. Decision Tree	99 vs. 63	7.56 (0.0058)	13 vs. 6	1.89 (0.170)
Random Forest vs. RBF-SVM	135 vs. 55	32.85 ($< 10^{-8}$)	13 vs. 9	0.41 (0.520)
Logistic Reg. vs. Random Forest	22 vs. 172	114.44 ($< 10^{-29}$)	11 vs. 1	6.75 (0.0063)
Part B: PCA Subspace Ablation	**Accuracy (%)**	**Storm Recall (%)**	**F1-Score**	**ROC / PR-AUC**
Logistic Reg. (Full 22 Features)	**75.87**	**96.34**	**0.3740**	**0.9411 / 0.5478**
Logistic Reg. (7 PCA Modes, 91.5%)	74.77	93.29	0.3562	0.9287 / 0.5240
k-NN (Full 22 Features)	**93.16**	**28.66**	**0.3852**	**0.8219 / 0.3547**
k-NN (7 PCA Modes, 91.5%)	92.47	24.39	0.3265	0.8159 / 0.3053

C. Operational and Flood Case Study Results (RO3)

1) Cost-Sensitive Threshold Optimisation

Table X and Fig. 12 evaluate how adjusting the Random Forest warning threshold changes storm detection and total disaster cost under the 15-to-1 penalty rule (where each missed storm costs 15 points and each false alarm costs 1 point). At the standard 0.50 cutoff, Random Forest misses 16 extreme storms and produces 363 false alarms, resulting in a total 15-to-1 cost of 603. When we apply the **leak-free training cross-validation threshold** of 0.30 (chosen strictly using 1995–2018 training folds without viewing test data), holdout storm recall rises from 90.24% to **near-complete storm detection** (98.17% recall, catching 161 of 164 extreme rain events), cutting missed storms by more than 81% (from 16 down to just 3 missed storms) and lowering total disaster cost from 603 to 550. For comparison, the post-hoc test minimum occurs at a threshold of 0.40 (catching 157 of 164 storms with a cost of 532), which is reported only as a retrospective reference because finding it requires looking at test labels.

TABLE X
RANDOM FOREST COST-SENSITIVE THRESHOLD SWEEP UNDER THE 15-TO-1 MISSED-STORM PENALTY (2019–2024 TEST SET)

Probability Cutoff & Rule	**Acc (%)**	**Prec (%)**	**Recall (%)**	**F1**	**Caught / Missed**	**False Alarms**	**15:1 Cost**
0.10 (High Standby Watch)	61.59	16.30	100.00	0.2803	164 / 0	842	842
0.20 (Low Warning Cutoff)	70.44	20.12	99.39	0.3347	163 / 1	647	662
0.30 (Prospective Training CV)	**76.82**	**24.17**	**98.17**	**0.3880**	**161 / 3**	**505**	**550 (OOF)**
0.35 (Intermediate Cutoff)	78.38	25.16	95.73	0.3985	157 / 7	467	572
0.40 (Retrospective Test Min)	80.20	26.88	95.73	0.4198	157 / 7	427	532 (Test)
0.50 (Standard Default Cutoff)	82.71	28.96	90.24	0.4385	148 / 16	363	603
0.60 (Moderate Confidence)	84.72	31.13	85.98	0.4571	141 / 23	312	657
0.70 (High-Priority Red Alert)	87.96	36.19	79.88	**0.4981**	131 / 33	231	726

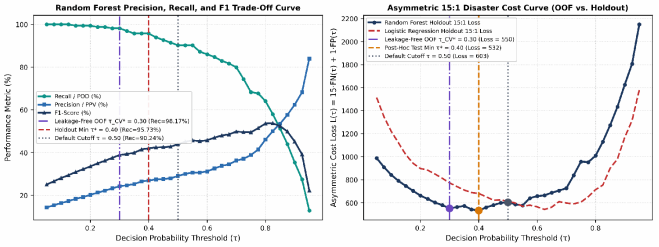


Fig. 12. Random Forest precision, recall, and F1 trade-off curves across probability cutoffs (left) and the 15-to-1 disaster cost curve comparing the training-derived 0.30 threshold against the 0.40 test minimum and 0.50 default cutoff (right).

2) Retrospective Validation on the September 2024 Flood

Fig. 13 tracks daily rainfall and model warning probabilities day by day across the catastrophic 20–30 September 2024 Kathmandu flood event [6, 7]. Following four days of light rain (0.39 to 1.14 mm/day during 20–23 September, where Random Forest probabilities remained low at 0.1831 or less), rainfall jumped suddenly to 47.54 mm/day on 24 September, 55.89 mm/day on 25 September, 16.56 mm/day on 26 September, 76.28 mm/day on 27 September, and 93.26 mm/day on 28 September. On the sudden first-day onset (24 September, when yesterday's rain was only 0.58 mm), same-day Logistic Regression immediately detected the spike in relative humidity (90.47%) and cloud cooling (daily temperature range compressed to 3.66°C), issuing a strong warning probability of 0.7154. From 25 to 28 September, once 1-day and 3-day rain memory engaged, both Random Forest (probabilities of 0.7189, 0.6992, 0.6973, and 0.6881) and Logistic Regression (probabilities of 0.9962, 0.9939, 0.8674, and 0.9997) issued sustained high-confidence flood alerts well above both the 0.30 and 0.50 thresholds.

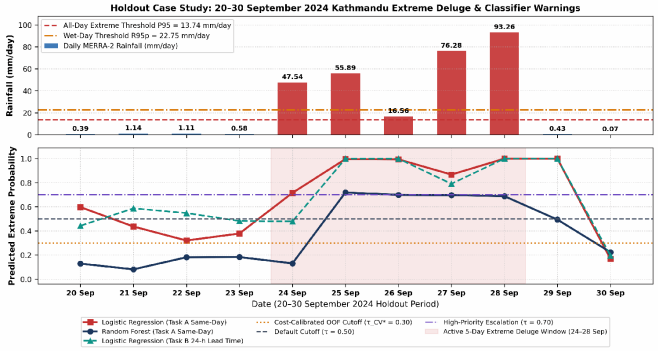


Fig. 13. Retrospective timeline of the 20–30 September 2024 Kathmandu flood disaster: daily rainfall against the 13.74 mm/day and 22.75 mm/day thresholds (top), and predicted storm probabilities from Random Forest and Logistic Regression (bottom).

3) 24-Hour Advance Forecasting and Wet-Day Benchmarks

Table XI and Fig. 14 compare model performance across Task A (same-day atmospheric state classification), Task B (strict 24-hour advance forecasting where all weather predictors come from yesterday or earlier), and Task C (severe wet-day deluge classification above 22.75 mm/day). Under the strict 24-hour lead time (Task B), Random Forest maintains the highest balanced F1-score (0.4097), catching 144 of 164 extreme storms a full day in advance (87.80% recall, ROC-AUC of 0.9135, Precision-Recall AUC of 0.4194), while Logistic Regression catches 152 of 164 storms 24 hours ahead (92.68% recall, ROC-AUC of 0.9088, Precision-Recall AUC of 0.4526). In contrast, k-NN recall drops to just 17.68% on Task B (missing 135 of 164 storms). On Task C (wet-day threshold of 22.75 mm/day across 64 test deluges), Logistic Regression detects 62 of 64 severe deluges (96.88% recall, ROC-AUC of 0.9461) and Random Forest detects 57 of 64 (89.06% recall, ROC-AUC of 0.9276), confirming robust early-warning skill across both lead times and severity thresholds.

TABLE XI
COMPARISON OF SAME-DAY CLASSIFICATION (TASK A), STRICT 24-HOUR ADVANCE FORECASTING (TASK B), AND WET-DAY DELUGE DETECTION (TASK C)

Forecasting Task & Model	Acc (%)	Prec (%)	Recall (%)	F1	ROC-AUC	PR-AUC	TP / FN / FP
Task A (Same-Day, 13.74 mm) – RF	82.71	28.96	90.24	**0.4385**	0.9349	**0.5517**	148 / 16 / 363
Task A (Same-Day, 13.74 mm) – LR	75.87	23.20	**96.34**	0.3740	**0.9411**	0.5478	158 / 6 / 523
Task B (24-Hr Lead, 13.74 mm) – RF	81.07	26.72	87.80	**0.4097**	**0.9135**	0.4194	144 / 20 / 395
Task B (24-Hr Lead, 13.74 mm) – LR	72.31	20.35	**92.68**	0.3337	0.9088	**0.4526**	152 / 12 / 595
Task B (24-Hr Lead, 13.74 mm) – k-NN	**92.29**	**46.03**	17.68	0.2555	0.7812	0.2509	29 / 135 / 34
Task C (Wet-Day, 22.75 mm) – RF	83.85	14.11	89.06	**0.2436**	0.9276	0.2851	57 / 7 / 347
Task C (Wet-Day, 22.75 mm) – LR	78.01	11.44	**96.88**	0.2046	**0.9461**	**0.3985**	62 / 2 / 480

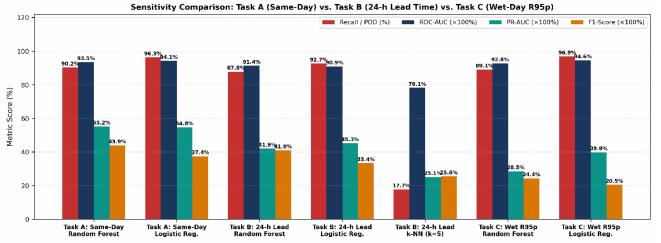


Fig. 14. Holdout performance comparison across Task A (same-day classification at 13.74 mm/day), Task B (strict 24-hour advance forecasting), and Task C (severe wet-day classification at 22.75 mm/day).

4) *Practical Comparison Across All Three Operational Tasks*

Comparing Tasks A, B, and C side by side demonstrates that model rankings remain consistent across different warning horizons and rainfall thresholds. Across every task, Random Forest and Logistic Regression form the top-performing pair, whereas unweighted k-NN consistently ranks last in storm detection. Moving from same-day diagnosis (Task A) to 24-hour advance forecasting (Task B) lowers Random Forest recall by only 2.44 percentage points (from 90.24% to 87.80%, or 4 additional missed storms over 6 years) and lowers Logistic Regression recall by only 3.66 percentage points (from 96.34% to 92.68%). This small drop confirms that multi-day valley moisture build-up and 3-to-7-day past rainfall totals carry strong predictive memory a full day before heavy rain arrives.

VI. Discussion and Conclusions

A. Why PCA Hurts Recall and Why k-NN Fails

1) Effect of PCA Compression on Storm Signals

A key finding linking RO1 and RO2 is that while PCA compresses the 22 weather predictors into 7 components capturing 91.46% of overall variance, training supervised models on those 7 components reduces storm recall by 3.05 percentage points for Logistic Regression and 4.27 percentage points for k-NN (Table IX). The practical reason is straightforward: PCA ranks components by day-to-day seasonal variation across the entire year (which is dominated by the smooth shift between dry winters and warm summers) rather than rare one-day storm spikes. Blending all 22 variables into 7 combined axes blurs sharp single-variable warning signs—such as a sudden jump in yesterday's rainfall or a sharp drop in daily temperature range—that classifiers rely on to spot extreme cloudbursts [16, 17].

2) Why Unweighted k-NN Suffers the Accuracy Paradox

Comparing the five classifiers refutes claims in earlier urban rainfall studies [2] that unweighted k-NN is suitable for rainfall prediction. On the 2019–2024 Kathmandu test set, k-NN achieves 93.16% overall accuracy but misses 71.34% of same-day extreme storms (Table VII) and 82.32% of next-day storms (Table XI). Because k-NN measures equal straight-line distances across all 22 features without weighting rare storm days, background variation in wind and temperature scatters storm days across the feature space. Inside the wet monsoon regime (Cluster 2), where 87.35% of days are non-extreme, the 5 nearest neighbours of a developing storm are almost always ordinary monsoon days, causing k-NN to vote "non-extreme" during major floods.

3) Complementary Strengths of Random Forest and Logistic Regression

In contrast, Random Forest and class-weighted Logistic Regression solve two different operational problems. A single CART tree over-relies on yesterday's rainfall (73.35% importance, Table VIII), acting mainly as a persistence detector that struggles when a storm begins abruptly after dry weather. Random Forest fixes this single-variable trap by forcing each tree split to choose from 4 random features, spreading attention across 3-day rain accumulation, specific humidity, dew point, and cloud cooling to eliminate 160 false alarms compared with Logistic Regression. Meanwhile, class-weighted Logistic Regression acts as an ultra-sensitive moisture detector: on 24 September 2024, when prior rainfall was near zero, Logistic Regression immediately flagged the sudden humidity and cloud-cover jump (probability 0.7154), whereas Random Forest locked onto the multi-day deluge from 25 to 28 September.

B. Three-Tier Operational Flood Warning Protocol

1) Yellow Tier: Internal Forecaster Watch

To turn these complementary model strengths into a practical workflow for meteorologists at Tribhuvan International Airport without causing public false-alarm fatigue, the Yellow Tier is triggered 24 hours in advance when Task B Logistic Regression probability reaches 0.50 (catching 92.68% of storms a day ahead) or on the same day when Task A Random Forest reaches the 0.30 training threshold (catching 98.17% of storms, 161 of 164 events). This tier alerts duty forecasters to monitor radar and river gauges internally, where false alarms carry minimal public cost.

2) Orange Tier: Municipal Preparedness Standby

The Orange Tier is triggered when same-day Random Forest probability reaches 0.40 or Logistic Regression reaches 0.70 (catching 95.73% of storms and immediately flagging the 24 September 2024 onset), prompting municipal crews to clear drainage channels and place emergency teams on standby.

3) Red Tier: High-Priority Flood Escalation

The Red Tier is triggered when Random Forest exceeds 0.68 and Logistic Regression exceeds 0.85 simultaneously (capturing the 25–28 September 2024 flood peak) and is confirmed by real-time ground rain gauges and Bagmati river-level sensors.

4) Practical Guidance for Operational Forecasters

In daily airport and municipal operations, this three-tier structure prevents warning fatigue by separating internal staff preparation from public alarms. Because Yellow Tier watches stay inside the meteorological office, forecasters can use the sensitive 0.30 Random Forest threshold and 24-hour Logistic Regression alerts to catch 98.17% of potential storms without causing unnecessary public disruption on humid monsoon break days.

Public and municipal escalations occur only when probabilities cross the higher Orange and Red thresholds alongside ground-gauge confirmation.

C. Ethical, Legal, and Study Limitations

1) Social, Ethical, and Legal Responsibilities

Deploying machine learning for flood early warning carries serious social, ethical, legal, and professional responsibilities under the British Computer Society Code of Conduct. In the Kathmandu Valley, flood damage falls heaviest on vulnerable low-income riverside communities (*Sukumbasi* settlements) along the Bagmati and Bishnumati corridors [6, 7]. Relying on an accuracy-optimised model like unweighted k-NN (which misses over 71% of floods) would leave communities unwarned, while issuing unverified public alarms would erode public trust. Under Nepal's *Disaster Risk Reduction and Management Act, 2074 (2017)*, legal authority to issue public flood warnings belongs solely to the Department of Hydrology and Meteorology (DHM) and the National Disaster Risk Reduction and Management Authority [6]. Therefore, our models and interactive dashboard (Appendix D) are designed strictly as human-in-the-loop decision-support tools for qualified hydrologists.

2) Methodological and Physical Study Limitations

Four practical study limitations must also be noted. First, a single 50 km MERRA-2 reanalysis grid cell averages rainfall over a wide area, recording 93.26 mm/day on 28 September 2024 whereas point ground gauges recorded over 240 mm [6, 7]. Second, daily totals do not capture 1-hour cloudburst intensity. Third, Task A diagnoses same-day weather states whereas Task B provides true 24-hour advance forecasts with slightly higher false alarms. Fourth, surface variables do not include upper-atmosphere wind divergence or river-channel water routing.

D. Conclusions and Achievement of Research Objectives (RO1–RO3)

1) Achievement of RO1: Data and Unsupervised Regime Analysis

A 30-year daily dataset (10,951 complete post-lag records) was engineered with 22 physical weather features, handling post-2012 calm-wind codes cleanly, shifting rolling rainfall windows strictly to past days, and splitting chronologically at 31 December 2018 (8,759 training days and 2,192 holdout test days) around training-only thresholds (13.74 mm/day all-day and 22.75 mm/day wet-day). Unsupervised PCA retained 91.46% of variance in 7 components, FastICA isolated non-normal storm bursts, and K-Means (3 clusters, Silhouette 0.3119) identified an Active Saturated Monsoon regime containing 94.99% (417 of 439) of training storms.

2) Achievement of RO2: Comparative Classifier Evaluation

Across five supervised classifiers, Random Forest achieved the best balanced test performance (F1 of 0.4385, Precision-Recall AUC of 0.5517, ROC-AUC of 0.9349, and 90.24% recall) by overcoming CART's 73.35% single-variable reliance on yesterday's rain. Class-weighted Logistic Regression achieved the highest storm recall (96.34%, 158 of 164 events, ROC-AUC of 0.9411), RBF-SVM achieved top probability calibration (Brier 0.0553) at higher computing cost (10.45 s), and unweighted k-NN exposed the accuracy paradox (93.16% accuracy but 28.66% storm recall).

3) Achievement of RO3: Operational and Cost-Sensitive Validation

Applying the training-derived cost-sensitive threshold of 0.30 under a 15-to-1 missed-flood penalty raised Random Forest test recall to 98.17% (161 of 164 storms caught, cutting missed floods from 16 to 3 and lowering cost from 603 to 550). The framework maintained strong skill at a strict 24-hour lead time (Task B: Random Forest recall 87.80%, Logistic Regression recall 92.68%) and on severe wet-day deluges above 22.75 mm/day (Task C: Logistic Regression recall 96.88%), while retrospectively flagging both the 24 September 2024 onset and the 25–28 September Kathmandu flood peak.

4) Directions for Future Work

Future research should combine gridded MERRA-2 predictors with hourly DHM ground rain gauges and weather radar across the Kathmandu Valley rim, link calibrated storm probabilities with hydraulic river-stage models for the Bagmati basin, and incorporate upper-air moisture transport indicators.

References

[1] S. Poudyal, S. Katwal, and R. Shakya, "Rainfall prediction in Kathmandu City using machine learning and deep learning techniques, *Int. J. Eng. Technol. Infrastruct. Dev. (InJET-InDev)*, vol. 2, no. 2, pp. 18–25, May 2026, doi: 10.3126/injet-indev.v2i2.95697.

[2] T. Nurmayati, "Performance comparison of Random Forest, Support Vector Machine, and K-Nearest Neighbors algorithms in ten-minute rainfall prediction for urban flood mitigation in South Tangerang, *J. Appl. Inform. Comput. (JAIC)*, vol. 10, no. 4, pp. 3911–3919, Aug. 2026, doi: 10.30871/jaic.v10i4.10809.

[3] Z.-S. Liu, M. Boy, and R. Makkonen, "TailWeather: From tail to extremes, a global climatological dataset for machine-learning weather forecasting, *arXiv preprint* arXiv:2609.12496v1, Sep. 2026. [Online]. Available: https://arxiv.org/abs/2609.12496v1

[4] D. Chaulagain, R. L. Ray, A. O. Yakub, *et al.*, "Extreme precipitation dynamics and El Niño-Southern Oscillation influences in Kathmandu Valley, Nepal, *Water*, vol. 17, no. 9, p. 1397, May 2025, doi: 10.3390/w17091397.

[5] K. Khanal, B. Baniya, M. Sigdel, *et al.*, "Spatial and temporal variation of precipitation extreme in the Kathmandu Valley during last three decades, *J. Hydrol. Meteorol.*, vol. 12, no. 1, pp. 34–46, Dec. 2024, doi: 10.3126/jhm.v12i1.72653.

[6] Department of Hydrology and Meteorology (DHM), *Situational Report on Extreme Precipitation and Flooding Event of 27–29 September 2024*, Government of Nepal, Kathmandu, Nepal, Oct. 2024. [Online]. Available: https://www.dhm.gov.np/

[7] International Centre for Integrated Mountain Development (ICIMOD), "Death toll climbs as torrential rains pound Nepal, *ICIMOD Situational Analysis*, Kathmandu, Nepal, Sep. 2024. [Online]. Available: https://www.icimod.org/death-toll-climbs-as-torrential-rains-pound-nepal/

[8] S. Phuyal, "Precipitation in Kathmandu Valley: A spatial-temporal study, *Int. J. Environ. Clim. Change*, vol. 14, no. 9, pp. 266–278, Sep. 2024, doi: 10.9734/ijecc/2024/v14i94410.

[9] O. A. Wani, S. S. Mahdi, M. Yeasin, *et al.*, "Predicting rainfall using machine learning, deep learning, and time series models across an altitudinal gradient in the North-Western Himalayas, *Sci. Rep.*, vol. 14, Art. no. 27876, Nov. 2024, doi: 10.1038/s41598-024-77687-x.

[10] M. Pujara and N. Paudel, "Rainfall prediction using Long Short-Term Memory and Gated Recurrent Unit with various meteorological parameters, *Nepalese J. Stat.*, vol. 8, no. 1, pp. 47–60, 2024, doi: 10.3126/njs.v8i1.73165.

[11] B. Bashyal, D. Bhusal, K. Singh, and R. Koju, "Rainfall prediction using wavelet transform and transformers, *KEC J. Sci. Eng.*, vol. 9, no. 1, pp. 144–153, 2025, doi: 10.3126/kjse.v9i1.78379.

[12] B. Paneru and B. Paneru, "Real-time rainfall prediction in Kathmandu, Kapan area using sensor data with machine learning and linear regression, *J. Soft Comput. Paradigm*, vol. 5, no. 3, pp. 266–286, Sep. 2023, doi: 10.36548/jscp.2023.3.004.

[13] S. Sayago *et al.*, "Comparative evaluation of NASA POWER, ERA5, and observational data for accuracy and reliability, *Theor. Appl. Climatol.*, vol. 156, Jun. 2025, doi: 10.1007/s00704-025-05512-w.

[14] K. Bi, L. Xie, H. Zhang, X. Chen, X. Gu, and Q. Tian, "Accurate medium-range global weather forecasting with 3D neural networks, *Nature*, vol. 619, no. 7970, pp. 533–538, Jul. 2023, doi: 10.1038/s41586-023-06185-3.

[15] R. Lam, A. Sanchez-Gonzalez, M. Willson, *et al.*, "Learning skillful medium-range global weather forecasting, *Science*, vol. 382, no. 6677, pp. 1416–1421, Dec. 2023, doi: 10.1126/science.adi2336.

[16] T. Talan, "Machine learning-based rainfall prediction across temporal scales: Model benchmarking and explainability analysis, *Stoch. Environ. Res. Risk Assess.*, vol. 40, Art. no. 100, 2026, doi: 10.1007/s00477-026-03245-8.

[17] Z. Zhang, Y. Liu, and H. Chen, "Identification and temporal distribution of typical rainfall types using PCA dimension reduction and K-means++ clustering, *Hydrology*, vol. 12, no. 4, p. 88, Apr. 2025, doi: 10.3390/hydrology12040088.

[18] Y. Wu, Z. Zhang, X. Qi, W. Hu, and S. Si, "Prediction of flood sensitivity based on Logistic Regression, eXtreme Gradient Boosting, and Random Forest modeling methods, *Water Sci. Technol.*, vol. 89, no. 10, pp. 2605–2624, May 2024, doi: 10.2166/wst.2024.146.

[19] X. Liu *et al.*, "Influencing factors and risk assessment of precipitation-induced flooding in Zhengzhou, China, based on Random Forest and XGBoost algorithms, *Int. J. Environ. Res. Public Health*, vol. 19, no. 24, p. 16544, Dec. 2022, doi: 10.3390/ijerph192416544.

[20] S. M. Toufique, S. U. Bhuiyan, A. Lateef, A. Zaman, J. Bin Islam, and D. Z. Karim, "Implementing machine learning techniques to forecast floods in Bangladesh, in *Proc. 2024 Int. Conf. Electr., Comput. Energy Technol. (ICECET)*, IEEE, Jul. 2024, pp. 1–6, doi: 10.1109/ICECET61485.2024.10698703.

APPENDIX

GitHub Link: https://github.com/gurungbishow/STW7072CEM-Machine-Learning.git
Video Link: https://youtu.be/1XqPLvwBD_4

A. DATA PREPROCESSING AND FEATURE ENGINEERING IMPLEMENTATION

data_processing.py > ...

```python
import os
import json
import numpy as np
import pandas as pd

FEATURE_COLS_TASK_A = [
    "T2M", "T2M_MAX", "T2M_MIN", "temp_diurnal_range",
    "T2MDEW", "dew_point_depression", "RH2M", "QV2M",
    "PS", "pressure_tendency", "WS10M", "WS10M_MAX",
    "ALLSKY_SFC_SW_DWN", "TS", "wind_calm_flag",
    "precip_lag1", "precip_lag2", "precip_lag3",
    "precip_roll3", "precip_roll7",
    "sin_doy", "cos_doy",
]

FEATURE_COLS_TASK_B = [
    "T2M_lag1", "T2M_MAX_lag1", "T2M_MIN_lag1", "temp_diurnal_range_lag1",
    "T2MDEW_lag1", "dew_point_depression_lag1", "RH2M_lag1", "QV2M_lag1",
    "PS_lag1", "pressure_tendency_lag1", "WS10M_lag1", "WS10M_MAX_lag1",
    "ALLSKY_SFC_SW_DWN_lag1", "TS_lag1", "wind_calm_flag_lag1",
    "precip_lag1", "precip_lag2", "precip_lag3",
    "precip_roll3", "precip_roll7",
    "sin_doy", "cos_doy",
]


def resolve_dataset_path(csv_path=None):
    """Resolves the Kathmandu NASA POWER MERRA-2 CSV path across working
    directories."""
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    candidates = []
    if csv_path:
        candidates.append(csv_path)
    candidates.extend([
        os.path.join(base_dir, "data", "kathmandu_merra2_1995_2024.csv"),
        os.path.join(base_dir, "data",
        "kathmandu_airport_daily_meteorology_1995_2024.csv"),
        "data/kathmandu_merra2_1995_2024.csv",
        "data/kathmandu_airport_daily_meteorology_1995_2024.csv",
        "../data/kathmandu_merra2_1995_2024.csv",
        "../data/kathmandu_airport_daily_meteorology_1995_2024.csv",
    ])
    for path in candidates:
        if os.path.exists(path):
            return os.path.abspath(path)
    raise FileNotFoundError(f"Could not locate Kathmandu dataset CSV in
    candidates: {candidates}")
```

data_processing.py > ...

```python
def audit_raw_dataset(csv_path=None):
    audit_summary = {
        "is_monotonic_increasing": bool(dates.is_monotonic_increasing),
        "duplicate_date_count": int(dates.duplicated().sum()),
        "missing_calendar_days": int(len(set(expected_dates) - set(dates))),
        "nan_values_total": int(df.isnull().sum().sum()),
        "fill_value_minus999_total": int((df[raw_vars] == -999.0).sum().sum
        ()),
        "negative_precip_days": int((df["PRECTOTCORR"] < 0).sum()),
        "max_precip_mm": float(df["PRECTOTCORR"].max()),
        "max_precip_date": str(df.loc[df["PRECTOTCORR"].idxmax(), "date"]),
        "sep28_2024_precip_mm": float(df.loc[df["date"] == "2024-09-28",
        "PRECTOTCORR"].values[0]),
        "first_7_days_precip_mm": [float(v) for v in df["PRECTOTCORR"].iloc
        [:7].tolist()],
        "wd10m_negative_calm_days_all": int((df["WD10M"] < 0).sum()),
        "wd10m_valid_days_train_prelag": int((train_raw["WD10M"] >= 0).sum
        ()),
        "wd10m_valid_days_train_postlag": int((train_post["WD10M"] >= 0).sum
        ()),
        "wd10m_negative_days_test": int((df[dates.dt.year > 2018]["WD10M"]
        < 0).sum()),
        "variable_training_stats": var_stats,
    }
    return audit_summary


def load_and_preprocess_data(csv_path=None):
    """
    Loads raw NASA POWER daily meteorological data for the Kathmandu
    Airport grid cell,
    encodes negative WD10M calm-wind flags (-8994.0 / <0) via
    wind_calm_flag, and engineers
    22 physically grounded predictors for Task A (same-day atmospheric
    state classification)
    plus strictly lagged (t-1) predictors for Task B (24-hour lead-time
    forecasting).
    """
    resolved_path = resolve_dataset_path(csv_path)
    df = pd.read_csv(resolved_path)
    df["date"] = pd.to_datetime(df["date"])
    df = df.sort_values("date").reset_index(drop=True)

    # 1. Wind direction flag handling (negative values denote calm/
    undefined valley wind direction)
    df["wind_calm_flag"] = (df["WD10M"] < 0).astype(int)
    df["WD10M_clean"] = df["WD10M"].where(df["WD10M"] >= 0, np.nan)
```

data_processing.py > ...

```python
def audit_raw_dataset(csv_path=None):
    """
    Audits raw daily NASA POWER / MERRA-2 observations (1995-2024) for data
    quality,
    continuity, missingness, sentinel values, and variable summary
    statistics.
    """
    resolved_path = resolve_dataset_path(csv_path)
    df = pd.read_csv(resolved_path)
    dates = pd.to_datetime(df["date"])
    expected_dates = pd.date_range("1995-01-01", "2024-12-31", freq="D")

    raw_vars = [
        "PRECTOTCORR", "T2M", "T2M_MAX", "T2M_MIN", "T2MDEW",
        "RH2M", "QV2M", "PS", "WS10M", "WS10M_MAX",
        "WD10M", "ALLSKY_SFC_SW_DWN", "TS",
    ]

    train_raw = df[dates.dt.year <= 2018].copy()
    train_post = df.iloc[7:][dates.iloc[7:].dt.year <= 2018].copy()

    var_stats = {}
    for col in raw_vars:
        if col == "WD10M":
            s_pre = train_raw.loc[train_raw["WD10M"] >= 0, "WD10M"]
            s_post = train_post.loc[train_post["WD10M"] >= 0, "WD10M"]
        else:
            s_pre = train_raw[col]
            s_post = train_post[col]
        var_stats[col] = {
            "train_prelag_mean": round(float(s_pre.mean()), 4),
            "train_prelag_std": round(float(s_pre.std(ddof=1)), 4),
            "train_postlag_mean": round(float(s_post.mean()), 4),
            "train_postlag_std": round(float(s_post.std(ddof=1)), 4),
            "train_min": round(float(s_post.min()), 4),
            "train_median": round(float(s_post.median()), 4),
            "train_iqr": round(float(np.percentile(s_post, 75) - np.
            percentile(s_post, 25)), 4),
            "train_max": round(float(s_post.max()), 4),
        }

    audit_summary = {
        "resolved_csv_path": resolved_path,
        "raw_total_days": int(len(df)),
        "raw_columns_count": int(df.shape[1]),
        "date_start": str(df["date"].min()),
        "date_end": str(df["date"].max()),
```

data_processing.py > ...

```python
def load_and_preprocess_data(csv_path=None):
    # 2. Antecedent rainfall memory (strictly shifted to t-1..t-7 so day-t
    rain never leaks into X)
    df["precip_lag1"] = df["PRECTOTCORR"].shift(1)
    df["precip_lag2"] = df["PRECTOTCORR"].shift(2)
    df["precip_lag3"] = df["PRECTOTCORR"].shift(3)
    df["precip_roll3"] = df["PRECTOTCORR"].shift(1).rolling(3).sum()
    df["precip_roll7"] = df["PRECTOTCORR"].shift(1).rolling(7).sum()

    # 3. Thermodynamic & synoptic stability indicators
    df["temp_diurnal_range"] = df["T2M_MAX"] - df["T2M_MIN"]
    df["dew_point_depression"] = df["T2M"] - df["T2MDEW"]
    df["pressure_tendency"] = df["PS"] - df["PS"].shift(1)

    # 4. Seasonal cyclical harmonics (capturing the South Asian Summer
    Monsoon cycle)
    doy = df["date"].dt.day_of_year if hasattr(df["date"].dt,
    "day_of_year") else df["date"].dt.dayofyear
    df["sin_doy"] = np.sin(2.0 * np.pi * doy / 365.25)
    df["cos_doy"] = np.cos(2.0 * np.pi * doy / 365.25)

    # 5. Strictly lagged atmospheric predictors (t-1) for Task B 24-hour
    lead-time forecasting
    day_t_vars = [
        "T2M", "T2M_MAX", "T2M_MIN", "temp_diurnal_range",
        "T2MDEW", "dew_point_depression", "RH2M", "QV2M",
        "PS", "pressure_tendency", "WS10M", "WS10M_MAX",
        "ALLSKY_SFC_SW_DWN", "TS", "wind_calm_flag",
    ]
    for col in day_t_vars:
        df[f"{col}_lag1"] = df[col].shift(1)

    # Trim initial 7 rows (1995-01-01 to 1995-01-07) to initialize 7-day
    rolling window without NaNs
    df_clean = df.iloc[7:].copy().reset_index(drop=True)
    return df_clean


def split_and_construct_target(df, train_end_year=2018, percentile=95,
wet_day_only=False):
    """
    Strict chronological train-test split to prevent temporal lookahead
    leakage.
    Both extreme thresholds (all-day P95 = 13.7400 mm/day and wet-day R95p
    = 22.7520 mm/day
    for wet days R >= 1.0 mm/day) are computed STRICTLY on the 1995-2018
    training partition.
    """
```

data_processing.py > ...

```
def split_and_construct_target(df, train_end_year=2018, percentile=95,
    train_mask = df["date"].dt.year <= train_end_year
    test_mask = df["date"].dt.year > train_end_year

    train_precip = df.loc[train_mask, "PRECTOTCORR"]
    all_day_threshold = float(np.percentile(train_precip, percentile))
    wet_threshold = float(np.percentile(train_precip[train_precip >= 1.0],
    percentile))

    threshold = wet_threshold if wet_day_only else all_day_threshold

    df["target_extreme"] = (df["PRECTOTCORR"] >= all_day_threshold).astype
    (int)
    df["target_extreme_wet95"] = (df["PRECTOTCORR"] >= wet_threshold).astype
    (int)

    target_col = "target_extreme_wet95" if wet_day_only else
    "target_extreme"

    X_train = df.loc[train_mask, FEATURE_COLS_TASK_A].copy()
    y_train = df.loc[train_mask, target_col].copy()

    X_test = df.loc[test_mask, FEATURE_COLS_TASK_A].copy()
    y_test = df.loc[test_mask, target_col].copy()

    dates_train = df.loc[train_mask, "date"].copy()
    dates_test = df.loc[test_mask, "date"].copy()

    train_wet_days = int((df.loc[train_mask, "PRECTOTCORR"] >= 1.0).sum())
    test_wet_days = int((df.loc[test_mask, "PRECTOTCORR"] >= 1.0).sum())
    train_wet_ext = int(df.loc[train_mask, "target_extreme_wet95"].sum())
    test_wet_ext = int(df.loc[test_mask, "target_extreme_wet95"].sum())

    split_info = {
        "train_start": str(dates_train.min().date()),
        "train_end": str(dates_train.max().date()),
        "test_start": str(dates_test.min().date()),
        "test_end": str(dates_test.max().date()),
        "threshold_mm_exact": float(all_day_threshold),
        "threshold_mm": round(all_day_threshold, 2),
        "wet_day_r95p_threshold_mm_exact": float(wet_threshold),
        "wet_day_r95p_threshold_mm": round(wet_threshold, 2),
        "percentile": percentile,
        "raw_total_samples": 10958,
        "post_lag_total_samples": int(len(df)),
        "train_samples": int(len(y_train)),
        "test_samples": int(len(y_test)),
```

data_processing.py > ...

```
def split_and_construct_target(df, train_end_year=2018, percentile=95,
    split_info = {
        "train_positives": int(df.loc[train_mask, "target_extreme"].sum()),
        "train_negatives": int((df.loc[train_mask, "target_extreme"] == 0).
        sum()),
        "train_prevalence": float(df.loc[train_mask, "target_extreme"].mean
        ()),
        "test_positives": int(df.loc[test_mask, "target_extreme"].sum()),
        "test_negatives": int((df.loc[test_mask, "target_extreme"] == 0).sum
        ()),
        "test_prevalence": float(df.loc[test_mask, "target_extreme"].mean
        ()),
        "train_wet_days_ge_1mm": train_wet_days,
        "test_wet_days_ge_1mm": test_wet_days,
        "train_wet95_positives": train_wet_ext,
        "train_wet95_prevalence_all_days": float(train_wet_ext / len
        (y_train)),
        "train_wet95_prevalence_wet_days": float(train_wet_ext / max(1,
        train_wet_days)),
        "test_wet95_positives": test_wet_ext,
        "test_wet95_prevalence_all_days": float(test_wet_ext / len(y_test)),
        "test_wet95_prevalence_wet_days": float(test_wet_ext / max(1,
        test_wet_days)),
        "feature_names": FEATURE_COLS_TASK_A,
        "feature_names_task_b": FEATURE_COLS_TASK_B,
    }

    return X_train, y_train, X_test, y_test, dates_train, dates_test,
    split_info


if __name__ == "__main__":
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    results_dir = os.path.join(base_dir, "results")
    os.makedirs(results_dir, exist_ok=True)

    audit = audit_raw_dataset()
    df_clean = load_and_preprocess_data()
    X_tr, y_tr, X_te, y_te, d_tr, d_te, split_info =
    split_and_construct_target(df_clean)

    with open(os.path.join(results_dir, "dataset_split_info.json"), "w",
    encoding="utf-8") as f:
        json.dump(split_info, f, indent=2)

    print("=" * 80)
    print("DATA PROCESSING & AUDIT VERIFICATION (src/data_processing.py)")
    print("=" * 80)
```

data_processing.py > ...

```
    print(f"Resolved Dataset Path : {audit['resolved_csv_path']}")
    print(f"Raw Observations      : {audit['raw_total_days']:,} days ({audit
    ['date_start']} to {audit['date_end']})")
    print(f"Missing / NaN / -999  : NaN={audit['nan_values_total']}, -999=
    {audit['fill_value_minus999_total']}, Gaps={audit
    ['missing_calendar_days']}")
    print(f"Post-Lag Observations : {split_info['post_lag_total_samples']
    :,} days (after 7-day rolling window init)")
    print(f"Training Partition    : {split_info['train_samples']:,} days
    ({split_info['train_start']} to {split_info['train_end']})")
    print(f"  - All-Day P95       : {split_info['threshold_mm_exact']:.4f}
    mm/day -> {split_info['train_positives']} extremes ({split_info
    ['train_prevalence']*100:.2f}%)")
    print(f"  - Wet-Day R95p      : {split_info
    ['wet_day_r95p_threshold_mm_exact']:.4f} mm/day -> {split_info
    ['train_wet95_positives']} extremes ({split_info
    ['train_wet95_prevalence_all_days']*100:.2f}%)")
    print(f"Testing Partition     : {split_info['test_samples']:,} days
    ({split_info['test_start']} to {split_info['test_end']})")
    print(f"  - All-Day P95 Ext   : {split_info['test_positives']} extremes
    ({split_info['test_prevalence']*100:.2f}%)")
    print(f"  - Wet-Day R95p Ext  : {split_info['test_wet95_positives']}
    extremes ({split_info['test_wet95_prevalence_all_days']*100:.2f}%)")
    print(f"Engineered Features   : {len(split_info['feature_names'])} Task
    A predictors | {len(split_info['feature_names_task_b'])} Task B
    predictors")
```

B. MODEL TRAINING AND EVALUATION IMPLEMENTATION

train_and_evaluate.py > ...

```python
import os
import sys
import json
import time
import warnings
import numpy as np
import pandas as pd

warnings.filterwarnings("ignore")

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA, FastICA
from sklearn.model_selection import TimeSeriesSplit, GridSearchCV
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    roc_auc_score, average_precision_score, confusion_matrix,
    brier_score_loss, silhouette_score, davies_bouldin_score,
)
from sklearn.inspection import permutation_importance
from scipy.stats import binomtest


def run_mcnemar_tests(y_true, preds_dict):
    """Computes paired McNemar chi-square statistics (with continuity
    correction) and exact binomial p-values."""
    yt = np.asarray(y_true, dtype=int)
    ext_mask = yt == 1
    pairs = [
        ("Random Forest", "k-Nearest Neighbours"),
        ("Logistic Regression", "k-Nearest Neighbours"),
        ("Random Forest", "Decision Tree"),
        ("Random Forest", "Support Vector Machine"),
        ("Logistic Regression", "Random Forest"),
    ]
    rows = []
    for m1, m2 in pairs:
        p1 = np.asarray(preds_dict[m1], dtype=int)
        p2 = np.asarray(preds_dict[m2], dtype=int)
        c1 = p1 == yt
        c2 = p2 == yt
```

train_and_evaluate.py > ...

```python
def run_mcnemar_tests(y_true, preds_dict):
        b_all = int((c1 & ~c2).sum())
        c_all = int((~c1 & c2).sum())
        chi2_all = float(((abs(b_all - c_all) - 1.0) ** 2) / max(1, b_all +
        c_all)) if (b_all + c_all) > 0 else 0.0
        pval_all = float(binomtest(min(b_all, c_all), n=max(1, b_all +
        c_all), p=0.5).pvalue) if (b_all + c_all) > 0 else 1.0

        e1 = p1[ext_mask] == 1
        e2 = p2[ext_mask] == 1
        b_ext = int((e1 & ~e2).sum())
        c_ext = int((~e1 & e2).sum())
        chi2_ext = float((abs(b_ext - c_ext) - 1.0) ** 2) / max(1, b_ext +
        c_ext)) if (b_ext + c_ext) > 0 else 0.0
        pval_ext = float(binomtest(min(b_ext, c_ext), n=max(1, b_ext +
        c_ext), p=0.5).pvalue) if (b_ext + c_ext) > 0 else 1.0

        rows.append({
            "pair": f"{m1} vs. {m2}",
            "b_all": b_all,
            "c_all": c_all,
            "chi2_all": round(chi2_all, 2),
            "p_all": pval_all,
            "b_ext": b_ext,
            "c_ext": c_ext,
            "chi2_ext": round(chi2_ext, 2),
            "p_ext": pval_ext,
        })
    return rows


def perform_unsupervised_analysis(X_train, y_train, feature_names):
    """
    Performs K-Means clustering, PCA, and FastICA dimensionality reduction
    strictly on the
    standardized 1995-2018 training partition (N = 8,759 days, 22 features).
    Records empirical fit and transform runtimes.
    """
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)

    # 1. Optimal K search (k = 2 to 6)
    k_range = list(range(2, 7))
    inertias = []
    sil_scores_full = []
    sil_scores_sub = []
    db_scores = []
```

train_and_evaluate.py > ...

```python
def perform_unsupervised_analysis(X_train, y_train, feature_names):
    for k in k_range:
        km = KMeans(n_clusters=k, random_state=42, n_init=10)
        km.fit(X_train_scaled)
        inertias.append(float(km.inertia_))
        sil_full = float(silhouette_score(X_train_scaled, km.labels_,
        sample_size=None, random_state=42))
        sil_sub = float(silhouette_score(X_train_scaled, km.labels_,
        sample_size=5000, random_state=42))
        db = float(davies_bouldin_score(X_train_scaled, km.labels_))
        sil_scores_full.append(sil_full)
        sil_scores_sub.append(sil_sub)
        db_scores.append(db)

    # Fit K-Means with optimal k=3 and measure runtime
    t0 = time.perf_counter()
    km_optimal = KMeans(n_clusters=3, random_state=42, n_init=10)
    train_clusters = km_optimal.fit_predict(X_train_scaled)
    km_runtime = float(time.perf_counter() - t0)

    cluster_df = pd.DataFrame({
        "cluster": train_clusters,
        "target_extreme": np.asarray(y_train, dtype=int),
    })
    cluster_stats = cluster_df.groupby("cluster").agg(
        total_days=("target_extreme", "count"),
        extreme_events=("target_extreme", "sum"),
        extreme_rate=("target_extreme", "mean"),
    ).reset_index()
    total_ext = float(np.sum(y_train))
    cluster_stats["share_of_extremes"] = cluster_stats["extreme_events"] /
    total_ext

    # Compute unscaled physical centroid means per cluster
    X_train_df = pd.DataFrame(X_train, columns=feature_names)
    X_train_df["cluster"] = train_clusters
    unscaled_centroids = X_train_df.groupby("cluster")[feature_names].mean
    ().to_dict(orient="index")

    # 2. PCA & FastICA Dimensionality Reduction (strictly on X_train_scaled)
    t0 = time.perf_counter()
    pca = PCA(n_components=22, random_state=42)
    pca.fit(X_train_scaled)
    pca_runtime = float(time.perf_counter() - t0)

    t0 = time.perf_counter()
    ica = FastICA(n_components=7, random_state=42, max_iter=500)
    ica_coords = ica.fit_transform(X_train_scaled)
```

train_and_evaluate.py > ...

```python
def run_hyperparameter_tuning(X_train_scaled, y_train, X_train_raw):
    tscv = TimeSeriesSplit(n_splits=5)
    X_raw_arr = np.asarray(X_train_raw, dtype=np.float64)
    y_arr = np.asarray(y_train, dtype=int)

    # 1. Logistic Regression (L2-Regularized)
    pipe_lr = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(class_weight="balanced", solver="lbfgs",
        max_iter=1000, random_state=42)),
    ])
    param_lr = {"clf__C": [0.01, 0.1, 1.0, 10.0]}
    grid_lr = GridSearchCV(pipe_lr, param_lr, cv=tscv, scoring="f1",
    n_jobs=1)
    grid_lr.fit(X_raw_arr, y_arr)
    t0 = time.perf_counter()
    lr_best_C = grid_lr.best_params_["clf__C"]
    lr_final = LogisticRegression(C=lr_best_C, class_weight="balanced",
    solver="lbfgs", max_iter=1000, random_state=42)
    lr_final.fit(X_train_scaled, y_arr)
    lr_fit_time = float(time.perf_counter() - t0)

    # 2. Decision Tree (CART)
    param_dt = {
        "max_depth": [3, 4, 5, 6, 8],
        "min_samples_split": [10, 20, 50],
        "min_samples_leaf": [5, 10, 20],
    }
    grid_dt = GridSearchCV(
        DecisionTreeClassifier(class_weight="balanced", random_state=42),
        param_dt, cv=tscv, scoring="f1", n_jobs=1,
    )
    grid_dt.fit(X_raw_arr, y_arr)
    t0 = time.perf_counter()
    dt_final = grid_dt.best_estimator_
    dt_final.fit(X_raw_arr, y_arr)
    dt_fit_time = float(time.perf_counter() - t0)

    # 3. Support Vector Machine (RBF Kernel)
    pipe_svm = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", SVC(kernel="rbf", class_weight="balanced",
        probability=True, random_state=42)),
    ])
    param_svm = {"clf__C": [0.5, 1.0, 2.0], "clf__gamma": ["scale"]}
    grid_svm = GridSearchCV(pipe_svm, param_svm, cv=tscv, scoring="f1",
    n_jobs=1)
    grid_svm.fit(X_raw_arr, y_arr)
```

train_and_evaluate.py > ...

```python
def perform_unsupervised_analysis(X_train, y_train, feature_names):
    ica_runtime = float(time.perf_counter() - t0)

    ica_kurtosis = [
        float(np.mean((ica_coords[:, j] - np.mean(ica_coords[:, j])) **
        4) / (np.var(ica_coords[:, j]) ** 2) - 3.0)
        for j in range(ica_coords.shape[1])
    ]

    var_explained = [float(v) for v in pca.explained_variance_ratio_]
    eigenvalues = [float(v) for v in pca.explained_variance_]
    cum_var_explained = [float(v) for v in np.cumsum(pca.
    explained_variance_ratio_)]

    pca_loadings = pd.DataFrame(
        pca.components_[:3, :],
        columns=feature_names,
        index=["PC1", "PC2", "PC3"],
    )

    unsupervised_results = {
        "k_range": k_range,
        "inertias": inertias,
        "silhouette_scores": sil_scores_full,
        "silhouette_scores_subsample5000": sil_scores_sub,
        "davies_bouldin_scores": db_scores,
        "optimal_k": 3,
        "cluster_profiles": cluster_stats.to_dict(orient="records"),
        "cluster_centers_scaled": km_optimal.cluster_centers_.tolist(),
        "cluster_centers_unscaled": {str(k): v for k, v in
        unscaled_centroids.items()},
        "pca_eigenvalues": eigenvalues,
        "pca_var_explained": var_explained,
        "pca_cum_var_explained": cum_var_explained,
        "pca_components": pca_loadings.to_dict(),
        "ica_kurtosis": ica_kurtosis,
        "runtimes_sec": {
            "PCA": round(pca_runtime, 4),
            "FastICA": round(ica_runtime, 4),
            "KMeans_k3": round(km_runtime, 4),
        },
    }

    return unsupervised_results, km_optimal, pca, ica, scaler
```

```python
def run_hyperparameter_tuning(X_train_scaled, y_train, X_train_raw):
    """
```

train_and_evaluate.py > ...

```python
def run_hyperparameter_tuning(X_train_scaled, y_train, X_train_raw):
    t0 = time.perf_counter()
    svm_best_C = grid_svm.best_params_["clf__C"]
    svm_final = SVC(C=svm_best_C, gamma="scale", kernel="rbf",
    class_weight="balanced", probability=True, random_state=42)
    svm_final.fit(X_train_scaled, y_arr)
    svm_fit_time = float(time.perf_counter() - t0)

    # 4. Random Forest (Ensemble Bagging)
    param_rf = {
        "n_estimators": [100, 150],
        "max_depth": [4, 6, 8],
        "min_samples_leaf": [5, 10],
    }
    grid_rf = GridSearchCV(
        RandomForestClassifier(class_weight="balanced", random_state=42),
        param_rf, cv=tscv, scoring="f1", n_jobs=1,
    )
    grid_rf.fit(X_raw_arr, y_arr)
    t0 = time.perf_counter()
    rf_final = grid_rf.best_estimator_
    rf_final.fit(X_raw_arr, y_arr)
    rf_fit_time = float(time.perf_counter() - t0)

    # 5. k-Nearest Neighbours (Instance-Based)
    pipe_knn = Pipeline([
        ("scaler", StandardScaler()),
        ("clf", KNeighborsClassifier()),
    ])
    param_knn = {"clf__n_neighbors": [5, 9, 15, 21], "clf__weights":
    ["uniform", "distance"]}
    grid_knn = GridSearchCV(pipe_knn, param_knn, cv=tscv, scoring="f1",
    n_jobs=1)
    grid_knn.fit(X_raw_arr, y_arr)
    t0 = time.perf_counter()
    knn_best_k = grid_knn.best_params_["clf__n_neighbors"]
    knn_best_w = grid_knn.best_params_["clf__weights"]
    knn_final = KNeighborsClassifier(n_neighbors=knn_best_k,
    weights=knn_best_w)
    knn_final.fit(X_train_scaled, y_arr)
    knn_fit_time = float(time.perf_counter() - t0)

    def _get_std(g):
        if hasattr(g, "cv_std_"):
            return float(g.cv_std_)
        return float(g.cv_results_["std_test_score"][g.best_index_])
```

train_and_evaluate.py > ...

```python
def run_hyperparameter_tuning(X_train_scaled, y_train, X_train_raw):
    best_models = {
        "Logistic Regression": (
            lr_final,
            {"C": lr_best_C, "penalty": "l2", "solver": "lbfgs",
            "class_weight": "balanced"},
            float(grid_lr.best_score_),
            _get_std(grid_lr),
            lr_fit_time,
        ),
        "Decision Tree": (
            dt_final,
            {**grid_dt.best_params_, "class_weight": "balanced"},
            float(grid_dt.best_score_),
            _get_std(grid_dt),
            dt_fit_time,
        ),
        "Support Vector Machine": (
            svm_final,
            {"C": svm_best_C, "gamma": "scale", "class_weight": "balanced"},
            float(grid_svm.best_score_),
            _get_std(grid_svm),
            svm_fit_time,
        ),
        "Random Forest": (
            rf_final,
            {**grid_rf.best_params_, "class_weight": "balanced"},
            float(grid_rf.best_score_),
            _get_std(grid_rf),
            rf_fit_time,
        ),
        "k-Nearest Neighbours": (
            knn_final,
            {"n_neighbors": knn_best_k, "weights": knn_best_w},
            float(grid_knn.best_score_),
            _get_std(grid_knn),
            knn_fit_time,
        ),
    }

    return best_models


def select_oof_threshold_cv(X_train_raw, y_train, rf_params=None,
fn_weight=15, fp_weight=1):
    """
```

train_and_evaluate.py > ...

```python
def select_oof_threshold_cv(X_train_raw, y_train, rf_params=None,
        yp_lr = (oof_lr_p >= t).astype(int)
        tn_l, fp_l, fn_l, tp_l = confusion_matrix(oof_y, yp_lr).ravel()
        loss_lr = int(fn_weight * fn_l + fp_weight * fp_l)

        oof_sweep.append({
            "tau": t,
            "oof_rf_loss": loss_rf,
            "oof_rf_fn": int(fn),
            "oof_rf_fp": int(fp),
            "oof_rf_tp": int(tp),
            "oof_rf_tn": int(tn),
            "oof_rf_recall": float(tp / max(1, tp + fn)),
            "oof_rf_precision": float(tp / max(1, tp + fp)),
            "oof_rf_f1": float(2 * tp / max(1, 2 * tp + fp + fn)),
            "oof_lr_loss": loss_lr,
            "oof_lr_fn": int(fn_l),
            "oof_lr_fp": int(fp_l),
            "oof_lr_recall": float(tp_l / max(1, tp_l + fn_l)),
        })
        if loss_rf < best_loss_rf:
            best_loss_rf = loss_rf
            best_tau_rf = t

    return float(best_tau_rf), oof_sweep


def calculate_metrics(y_true, y_pred, y_prob):
    """Computes threshold-dependent and threshold-free classification
    metrics from arrays."""
    cm = confusion_matrix(y_true, y_pred)
    tn, fp, fn, tp = cm.ravel()

    acc = accuracy_score(y_true, y_pred)
    prec = precision_score(y_true, y_pred, zero_division=0)
    rec = recall_score(y_true, y_pred, zero_division=0)
    f1 = f1_score(y_true, y_pred, zero_division=0)
    f1_macro = f1_score(y_true, y_pred, average="macro", zero_division=0)
    spec = tn / (tn + fp) if (tn + fp) > 0 else 0.0
    roc_auc = roc_auc_score(y_true, y_prob)
    pr_auc = average_precision_score(y_true, y_prob)
    brier = brier_score_loss(y_true, y_prob)

    return {
        "Accuracy": float(acc),
        "Precision": float(prec),
        "Recall": float(rec),
        "F1 Score": float(f1),
```

train_and_evaluate.py > ...

```python
def select_oof_threshold_cv(X_train_raw, y_train, rf_params=None,
    """
    tscv = TimeSeriesSplit(n_splits=5)
    X_arr = np.asarray(X_train_raw, dtype=np.float64)
    y_arr = np.asarray(y_train, dtype=int)

    if rf_params is None:
        rf_params = {"n_estimators": 100, "max_depth": 8,
        "min_samples_leaf": 5}

    oof_rf_probs = []
    oof_lr_probs = []
    oof_targets = []

    for tr_idx, val_idx in tscv.split(X_arr):
        rf_fold = RandomForestClassifier(
            n_estimators=rf_params.get("n_estimators", 100),
            max_depth=rf_params.get("max_depth", 8),
            min_samples_leaf=rf_params.get("min_samples_leaf", 5),
            class_weight="balanced",
            random_state=42,
        )
        rf_fold.fit(X_arr[tr_idx], y_arr[tr_idx])
        oof_rf_probs.append(rf_fold.predict_proba(X_arr[val_idx])[:, 1])

        sc = StandardScaler().fit(X_arr[tr_idx])
        lr_fold = LogisticRegression(C=0.1, class_weight="balanced",
        solver="lbfgs", max_iter=1000, random_state=42)
        lr_fold.fit(sc.transform(X_arr[tr_idx]), y_arr[tr_idx])
        oof_lr_probs.append(lr_fold.predict_proba(sc.transform(X_arr
        [val_idx]))[:, 1])

        oof_targets.append(y_arr[val_idx])

    oof_rf_p = np.concatenate(oof_rf_probs)
    oof_lr_p = np.concatenate(oof_lr_probs)
    oof_y = np.concatenate(oof_targets)

    candidate_taus = [0.10, 0.15, 0.20, 0.25, 0.30, 0.35, 0.40, 0.45, 0.50,
    0.60, 0.70]
    best_tau_rf = None
    best_loss_rf = float("inf")
    oof_sweep = []

    for t in candidate_taus:
        yp_rf = (oof_rf_p >= t).astype(int)
        tn, fp, fn, tp = confusion_matrix(oof_y, yp_rf).ravel()
        loss_rf = int(fn_weight * fn + fp_weight * fp)
```

train_and_evaluate.py > ...

```python
def calculate_metrics(y_true, y_pred, y_prob):
        "F1_Macro": float(f1_macro),
        "Specificity": float(spec),
        "ROC_AUC": float(roc_auc),
        "PR_AUC": float(pr_auc),
        "Brier_Score": float(brier),
        "TN": int(tn),
        "FP": int(fp),
        "FN": int(fn),
        "TP": int(tp),
        "Event_Prevalence": float(np.mean(y_true)),
    }


def bootstrap_confidence_intervals(y_true, y_pred, y_prob,
n_bootstrap=1000, block_length=7, seed=42):
    """
    Computes genuine 95% circular moving-block bootstrap confidence
    intervals across
    n_bootstrap=1,000 resamples with block_length=7 days (matching the
    7-day synoptic
    persistence window) to preserve temporal autocorrelation. When
    block_length=1,
    reduces to standard i.i.d. bootstrap.
    """
    rng = np.random.RandomState(seed)
    y_true_arr = np.asarray(y_true, dtype=int)
    y_pred_arr = np.asarray(y_pred, dtype=int)
    y_prob_arr = np.asarray(y_prob, dtype=np.float64)
    n = len(y_true_arr)

    boot_metrics = {
        "Accuracy": [], "Precision": [], "Recall": [],
        "F1_Score": [], "ROC_AUC": [], "PR_AUC": [], "Brier_Score": [],
    }

    if block_length <= 1:
        for _ in range(n_bootstrap):
            idx = rng.randint(0, n, size=n)
            yt_s = y_true_arr[idx]
            if len(np.unique(yt_s)) < 2:
                continue
            yp_s = y_pred_arr[idx]
            pr_s = y_prob_arr[idx]
            boot_metrics["Accuracy"].append(accuracy_score(yt_s, yp_s))
            boot_metrics["Precision"].append(precision_score(yt_s, yp_s,
            zero_division=0))
```

train_and_evaluate.py > ...

```python
def bootstrap_confidence_intervals(y_true, y_pred, y_prob,
            boot_metrics["ROC_AUC"].append(roc_auc_score(yt_s, pr_s))
            boot_metrics["PR_AUC"].append(average_precision_score(yt_s,
            pr_s))
            boot_metrics["Brier_Score"].append(brier_score_loss(yt_s, pr_s))

    ci_results = {}
    for metric, values in boot_metrics.items():
        ci_lower = float(np.percentile(values, 2.5))
        ci_upper = float(np.percentile(values, 97.5))
        ci_mean = float(np.mean(values))
        ci_results[metric] = {
            "mean": ci_mean,
            "ci_lower": ci_lower,
            "ci_upper": ci_upper,
            "block_length": int(block_length),
            "n_bootstrap": int(n_bootstrap),
            "seed": int(seed),
            "str_format": f"{ci_mean*100:.2f}% [{ci_lower*100:.2f}%,
            {ci_upper*100:.2f}%]",
        }
    return ci_results


if __name__ == "__main__":
    from data_processing import load_and_preprocess_data,
    split_and_construct_target

    df_clean = load_and_preprocess_data()
    X_tr, y_tr, X_te, y_te, d_tr, d_te, split_info =
    split_and_construct_target(df_clean)
    unsup_res, km_m, pca_m, ica_m, scaler_m = perform_unsupervised_analysis
    (X_tr, y_tr, split_info["feature_names"])
    X_tr_s = scaler_m.transform(X_tr)
    X_te_s = scaler_m.transform(X_te)
    models = run_hyperparameter_tuning(X_tr_s, y_tr, X_tr)
    print("=" * 80)
    print("TRAINING & EVALUATION MODULE VERIFICATION (src/
    train_and_evaluate.py)")
    print("=" * 80)
    for name, (mdl, params, cv_f1, cv_std, fit_t) in models.items():
        X_ev = X_te.values if name in ["Decision Tree", "Random Forest"]
        else X_te_s
        yp = mdl.predict(X_ev)
        pr = mdl.predict_proba(X_ev)[:, 1]
        m = calculate_metrics(y_te, yp, pr)
        print(f"{name:<23} | CV F1={cv_f1:.4f}+/-{cv_std:.4f} | Test F1={m
        ['F1 Score']:.4f} | Rec={m['Recall']*100:.2f}% | ROC-AUC={m
```

C. *VISUAL ANALYTICS AND DIAGNOSTIC RESULTS*

generate_figures.py > ...

```python
import os
import sys
import json
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
import matplotlib.patches as patches
from matplotlib.gridspec import GridSpec

from data_processing import load_and_preprocess_data,
split_and_construct_target
from sklearn.metrics import (
    roc_curve, precision_recall_curve, confusion_matrix,
    brier_score_loss, roc_auc_score, average_precision_score,
)

plt.rcParams["font.family"] = "DejaVu Sans"
plt.rcParams["font.size"] = 9.5
plt.rcParams["axes.titlesize"] = 10.5
plt.rcParams["axes.labelsize"] = 9.5
plt.rcParams["xtick.labelsize"] = 8.5
plt.rcParams["ytick.labelsize"] = 8.5
plt.rcParams["legend.fontsize"] = 8.2
plt.rcParams["figure.dpi"] = 300
plt.rcParams["savefig.dpi"] = 300

NAVY = "#1B365D"
BLUE = "#2B6CB0"
TEAL = "#0D9488"
CORAL = "#C53030"
AMBER = "#D97706"
PURPLE = "#6B46C1"
SLATE = "#4A5568"


def generate_all_figures():
    base_dir = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    fig_dir = os.path.join(base_dir, "figures")
    res_dir = os.path.join(base_dir, "results")
    os.makedirs(fig_dir, exist_ok=True)

    verified_path = os.path.join(res_dir, "verified_metrics.json")
    npz_path = os.path.join(res_dir, "test_predictions.npz")
    if not os.path.exists(verified_path) or not os.path.exists(npz_path):
        from run_experiments import main as run_all_exp
        run_all_exp()
```

generate_figures.py > ...

```python
def generate_all_figures():
    if not os.path.exists(verified_path) or not os.path.exists(npz_path):
        from run_experiments import main as run_all_exp
        run_all_exp()

    with open(verified_path, "r", encoding="utf-8") as f:
        verified = json.load(f)
    d_npz = np.load(npz_path)

    df = load_and_preprocess_data()
    X_train, y_train, X_test, y_test, d_tr, d_te, split_info =
    split_and_construct_target(df)
    unsup_res = verified["unsupervised"]

    #
    ----------------------------------------------------------------------
    -
    # 1. figure_01_location.png: Study location & Kathmandu monthly
    climatology
    #
    ----------------------------------------------------------------------
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4), gridspec_kw=
    {"width_ratios": [1.05, 1.35]})
    ax1.set_facecolor("#EEF4F8")
    lon_c, lat_c = 85.359, 27.697
    rect = patches.Rectangle(
        (lon_c - 0.3125, lat_c - 0.25), 0.625, 0.50,
        linewidth=2.0, edgecolor=NAVY, facecolor="#90CDF4", alpha=0.38,
        label="MERRA-2 Grid Cell (0.5° × 0.625°)",
    )
    ax1.add_patch(rect)
    theta = np.linspace(0, 2 * np.pi, 200)
    ax1.plot(
        lon_c + 0.14 * np.cos(theta), lat_c + 0.10 * np.sin(theta),
        color=TEAL, linestyle="--", lw=1.6, label="Kathmandu Valley Rim (~2,
        000–2,800 m)",
    )
    ax1.scatter(
        [lon_c], [lat_c], color=CORAL, s=90, zorder=5, marker="^",
        label="Tribhuvan Intl. Airport (VNKT, 1,269.39 m)",
    )
    for name, dx, dy in [
        ("Shivapuri (2,732 m)", 0.0, 0.125),
        ("Phulchowki (2,762 m)", 0.08, -0.115),
        ("Chandragiri (2,551 m)", -0.12, -0.06),
    ]:
        ax1.annotate(name, (lon_c + dx, lat_c + dy), fontsize=7.5,
```

generate_figures.py > ...

```
def generate_all_figures():
    fontweight="bold", color=SLATE, ha="center")
ax1.set_xlim(lon_c - 0.45, lon_c + 0.45)
ax1.set_ylim(lat_c - 0.36, lat_c + 0.36)
ax1.set_xlabel("Longitude (°E)")
ax1.set_ylabel("Latitude (°N)")
ax1.set_title("Kathmandu Airport MERRA-2 Grid Domain",
fontweight="bold")
ax1.grid(True, linestyle=":", alpha=0.6)
ax1.legend(loc="lower left", frameon=True, facecolor="white",
framealpha=0.9, fontsize=7.5)

df["month"] = df["date"].dt.month
m_stats = df.groupby("month").agg(
    mean_rain=("PRECTOTCORR", "mean"),
    ext_count=("target_extreme", "sum"),
).reindex(range(1, 13))
months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun", "Jul", "Aug",
"Sep", "Oct", "Nov", "Dec"]
x_m = np.arange(1, 13)
ax2.bar(x_m - 0.18, m_stats["mean_rain"], width=0.36, color=BLUE,
alpha=0.88, label="Mean Daily Rainfall (mm/day)")
ax2.set_xlabel("Calendar Month (1995–2024)")
ax2.set_ylabel("Mean Daily Precipitation (mm/day)", color=BLUE,
fontweight="bold")
ax2.set_xticks(x_m)
ax2.set_xticklabels(months)
ax2.grid(True, axis="y", linestyle=":", alpha=0.6)
ax2_t = ax2.twinx()
ax2_t.bar(x_m + 0.18, m_stats["ext_count"], width=0.36, color=CORAL,
alpha=0.85, label="Extreme Days ≥ 13.74 mm (Count)")
ax2_t.set_ylabel("Total Extreme Rainfall Days (Count)", color=CORAL,
fontweight="bold")
ax2.set_title("Monthly Rainfall Climatology and Monsoon Extreme
Concentration", fontweight="bold")
h1, l1 = ax2.get_legend_handles_labels()
h2, l2 = ax2_t.get_legend_handles_labels()
ax2.legend(h1 + h2, l1 + l2, loc="upper left", frameon=True,
facecolor="white", fontsize=8)
plt.tight_layout()
plt.savefig(os.path.join(fig_dir, "figure_01_location.png"), dpi=300)
plt.close()

#
# ---------------------------------------------------------------------
-
# 2. figure_02_tail_distribution.png: Daily rainfall tail & P95 vs R95p
#
```

generate_figures.py > ...

```
def generate_all_figures():
    Intensification", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper right", frameon=True, facecolor="white",
    fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_02_tail_distribution.png"),
    dpi=300)
    plt.close()

    #
    # ---------------------------------------------------------------------
    -
    # 3. figure_03_workflow_cv.png: Chronological split & 5-fold
    TimeSeriesSplit
    #
    # ---------------------------------------------------------------------
    -
    tau_cv_star = verified["threshold_cost_sweep"]["tau_cv_star"]
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12.2, 4.8), gridspec_kw=
    {"width_ratios": [1.15, 1.2]})
    ax1.axis("off")
    ax1.set_xlim(0, 10)
    ax1.set_ylim(-0.35, 10.15)
    steps = [
        (5.0, 9.10, "1. Raw NASA POWER MERRA-2 Archive\n(1 Jan 1995 – 31
        Dec 2024, N = 10,958 Days)", "#E2E8F0", NAVY),
        (5.0, 7.36, "2. 7-Day Lag Initialization & Strict t−1 Rolling
        Shift\n(22 Engineered Predictors, N = 10,951 Post-Lag Days)",
        "#DBEAFE", NAVY),
        (5.0, 5.62, "3. Strict Chronological Split at 31 Dec 2018\nTrain:
        1995–2018 (8,759 d) | Holdout Test: 2019–2024 (2,192 d)",
        "#FEF3C7", "#92400E"),
        (5.0, 3.88, "4. Training-Only Quantile Thresholds & In-Fold
        Scaling\n(P95 = 13.74 mm/d, Wet-Day R95p = 22.75 mm/d, Pipeline
        Scaler)", "#D1FAE5", "#065F46"),
        (5.0, 2.14, f"5. 5-Fold Expanding-Window TimeSeriesSplit CV\n
        (Hyperparameter Tuning & Leakage-Free OOF τ*CV = {tau_cv_star:.2f})
        ", "#EDE9FE", "#5B21B6"),
        (5.0, 0.40, "6. Single Locked Evaluation on 2019–2024 Holdout Set\n
        (164 P95 Extremes, 7-Day Block-Bootstrap CIs, McNemar & Tasks B/C)
        ", "#FEE2E2", "#991B1B"),
    ]
    for x, y, label_txt, fc, ec in steps:
        box = patches.FancyBboxPatch(
            (0.25, y - 0.56), 9.5, 1.12, boxstyle="round,pad=0.08",
            facecolor=fc, edgecolor=ec, linewidth=1.6,
        )
```

generate_figures.py > ...

```
def generate_all_figures():
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.3))
    tr_rain = df.loc[df["date"].dt.year <= 2018, "PRECTOTCORR"].values
    te_rain = df.loc[df["date"].dt.year > 2018, "PRECTOTCORR"].values
    bins = np.linspace(0, 115, 60)
    ax1.hist(tr_rain, bins=bins, color=BLUE, alpha=0.72, label="Training
    (1995–2018, N=8,759)", edgecolor="white", lw=0.4)
    ax1.hist(te_rain, bins=bins, color=TEAL, alpha=0.68, label="Testing
    (2019–2024, N=2,192)", edgecolor="white", lw=0.4)
    ax1.axvline(13.74, color=CORAL, linestyle="--", lw=2.0, label="All-Day
    P95 = 13.74 mm/day (5.01% Train)")
    ax1.axvline(22.752, color=AMBER, linestyle="-.", lw=2.0, label="Wet-Day
    R95p = 22.75 mm/day (1.69% Train)")
    ax1.set_yscale("log")
    ax1.set_xlabel("Daily Precipitation (mm/day)")
    ax1.set_ylabel("Frequency (Log Scale, Days)")
    ax1.set_title("Right-Skewed Daily Rainfall Distribution (Log-Frequency)
    ", fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="upper right", frameon=True, facecolor="white",
    fontsize=8)

    r_grid = np.linspace(0.1, 95, 300)
    surv_tr = [np.mean(tr_rain >= r) * 100 for r in r_grid]
    surv_te = [np.mean(te_rain >= r) * 100 for r in r_grid]
    ax2.plot(r_grid, surv_tr, color=BLUE, lw=2.2, label="1995–2018 Training
    Exceedance (%)")
    ax2.plot(r_grid, surv_te, color=CORAL, lw=2.2, linestyle="-",
    label="2019–2024 Holdout Exceedance (%)")
    ax2.axvline(13.74, color=CORAL, linestyle="--", lw=1.6)
    ax2.axvline(22.752, color=AMBER, linestyle="-.", lw=1.6)
    ax2.annotate(
        "P95 = 13.74 mm/day\nTrain: 5.01% | Test: 7.48%",
        xy=(13.74, 7.48), xytext=(24, 14),
        arrowprops=dict(arrowstyle="->", color=CORAL, lw=1.2), fontsize=8,
        fontweight="bold", color=CORAL,
    )
    ax2.annotate(
        "Wet-Day R95p = 22.75 mm/day\nTrain: 1.69% | Test: 2.92%",
        xy=(22.752, 2.92), xytext=(36, 3.8),
        arrowprops=dict(arrowstyle="->", color=AMBER, lw=1.2), fontsize=8,
        fontweight="bold", color=AMBER,
    )
    ax2.set_yscale("log")
    ax2.set_xlabel("Precipitation Threshold r (mm/day)")
    ax2.set_ylabel("Exceedance Probability P(R ≥ r) [%]")
    ax2.set_title("Empirical Tail Exceedance Curve & Recent
```

generate_figures.py > ...

```
def generate_all_figures():
        ax1.add_patch(box)
        ax1.text(x, y, label_txt, ha="center", va="center", fontsize=7.8,
        fontweight="bold", color=ec, linespacing=1.22)
    for i in range(len(steps) - 1):
        ax1.annotate(
            "", xy=(5.0, steps[i + 1][1] + 0.64), xytext=(5.0, steps[i][1]
            - 0.64),
            arrowprops=dict(arrowstyle="->", lw=1.7, color=SLATE),
        )
    ax1.set_title("Leakage-Free Preprocessing & Pipeline Workflow",
    fontweight="bold", pad=8)

    n_tr = 8759
    n_te = 2192
    test_sz = n_tr // 6
    y_labels = ["Fold 1", "Fold 2", "Fold 3", "Fold 4", "Fold 5", "Final
    Holdout"]
    for i in range(5):
        tr_e = (i + 1) * test_sz
        val_sz = (n_tr - tr_e) if i == 4 else test_sz
        ax2.barh(i, tr_e, left=0, height=0.55, color=BLUE, alpha=0.85,
        label="In-Fold Train" if i == 0 else "")
        ax2.barh(i, val_sz, left=tr_e, height=0.55, color=AMBER, alpha=0.
        90, label="In-Fold Validation (~1,459 d)" if i == 0 else "")
    ax2.barh(5, n_tr, left=0, height=0.55, color=NAVY, alpha=0.90,
    label="Full Train (1995–2018, 8,759 d)")
    ax2.barh(5, n_te, left=n_tr, height=0.55, color=CORAL, alpha=0.90,
    label="Holdout Test (2019–2024, 2,192 d)")
    ax2.axvline(n_tr, color=CORAL, linestyle="--", lw=1.8)
    ax2.set_yticks(range(6))
    ax2.set_yticklabels(y_labels, fontweight="bold")
    ax2.invert_yaxis()
    ax2.set_xlabel("Chronological Sample Index (Days from 8 Jan 1995 to 31
    Dec 2024)")
    ax2.set_title("Expanding-Window 5-Fold TimeSeriesSplit & Holdout
    Design", fontweight="bold")
    ax2.grid(True, axis="x", linestyle=":", alpha=0.6)
    ax2.legend(loc="lower right", frameon=True, facecolor="white",
    fontsize=7.8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_03_workflow_cv.png"), dpi=300)
    plt.close()

    #
    # ---------------------------------------------------------------------
    -
```

generate_figures.py > ...

```
def generate_all_figures():
    # 4. figure_04_kmeans_diagnostics.png: K-Means Silhouette &
    Davies-Bouldin
    #
    ----------------------------------------------------------------------
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.2, 4.2))
    k_vals = unsup_res["k_range"]
    inertias = [v / 1000.0 for v in unsup_res["inertias"]]
    sils = unsup_res["silhouette_scores"]
    dbs = unsup_res["davies_bouldin_scores"]

    ax1.plot(k_vals, inertias, marker="o", lw=2.2, color=NAVY, label="WCSS
    Inertia (×10³)")
    ax1.set_xlabel("Number of Clusters (K)")
    ax1.set_ylabel("Within-Cluster Sum of Squares (×10³)", color=NAVY,
    fontweight="bold")
    ax1.set_xticks(k_vals)
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1_t = ax1.twinx()
    ax1_t.plot(k_vals, sils, marker="s", lw=2.2, linestyle="--",
    color=TEAL, label="Silhouette Score")
    ax1_t.set_ylabel("Mean Silhouette Coefficient", color=TEAL,
    fontweight="bold")
    ax1.axvline(3, color=CORAL, linestyle=":", lw=1.8, label="Selected K =
    3")
    ax1.set_title("WCSS Inertia Elbow & Silhouette Profile Across K ∈ {2..6}
    ", fontweight="bold")
    h1, l1 = ax1.get_legend_handles_labels()
    h2, l2 = ax1_t.get_legend_handles_labels()
    ax1.legend(h1 + h2, l1 + l2, loc="center right", frameon=True,
    facecolor="white", fontsize=8)

    ax2.plot(k_vals, dbs, marker="D", lw=2.2, color=CORAL,
    label="Davies-Bouldin Index (Lower is Better)")
    ax2.scatter([3], [dbs[1]], color=NAVY, s=90, zorder=5)
    ax2.annotate(
        f"Global Minimum at K = 3\n(DB = {dbs[1]:.4f}, Sil = {sils[1]:.4f})
        ",
        xy=(3, dbs[1]), xytext=(3.4, dbs[1] + 0.08),
        arrowprops=dict(arrowstyle="->", color=NAVY, lw=1.3),
        fontsize=8.5, fontweight="bold", color=NAVY,
    )
    ax2.set_xlabel("Number of Clusters (K)")
    ax2.set_ylabel("Davies-Bouldin Index", fontweight="bold")
    ax2.set_xticks(k_vals)
    ax2.set_title("Davies-Bouldin Cluster Separation Index Across K ∈ {2..6}
```

generate_figures.py > ...

```
def generate_all_figures():
    ax2.set_title("2D FastICA Non-Gaussian Source Projection (m = 7
    Subspace)", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper left", frameon=True, facecolor="white",
    fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_05_pca_ica_projections.png"),
    dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 6. figure_06_pca_scree_kmeans_radar.png: Combined PCA Scree & K-Means
    Radar
    #
    ----------------------------------------------------------------------
    -
    fig = plt.figure(figsize=(11.8, 4.5))
    gs = GridSpec(1, 2, width_ratios=[1.15, 1.0], figure=fig)
    ax1 = fig.add_subplot(gs[0])
    comps = np.arange(1, 16)
    var_pct = np.array(unsup_res["pca_var_explained"][:15]) * 100
    cum_pct = np.array(unsup_res["pca_cum_var_explained"][:15]) * 100
    ax1.bar(comps, var_pct, color=BLUE, alpha=0.85, label="Individual
    Explained Variance (%)")
    ax1.plot(comps, cum_pct, color=CORAL, marker="o", lw=2.2,
    label="Cumulative Explained Variance (%)")
    ax1.axhline(90.0, color=TEAL, linestyle="--", lw=1.6, label="90%
    Retention Threshold (m=7: 91.46%)")
    ax1.axvline(7, color=TEAL, linestyle=":", lw=1.6)
    ax1.set_xticks(comps)
    ax1.set_xlabel("Principal Component Index (m)")
    ax1.set_ylabel("Explained Variance (%)")
    ax1.set_ylim(0, 105)
    ax1.set_title("PCA Eigenvalue Scree Plot & Cumulative Variance
    Retention", fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="center right", frameon=True, facecolor="white",
    fontsize=8)

    ax2 = fig.add_subplot(gs[1], polar=True)
    radar_feats = ["QV2M", "RH2M", "T2MDEW", "precip_roll7", "T2M",
    "ALLSKY_SFC_SW_DWN", "temp_diurnal_range", "PS"]
    radar_labels = ["QV2M", "RH2M", "T2MDEW", "Roll7 Rain", "T2M",
    "Shortwave", "Diurnal Range", "Pressure"]
```

generate_figures.py > ...

```
def generate_all_figures():
    ", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper left", frameon=True, facecolor="white",
    fontsize=8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_04_kmeans_diagnostics.png"),
    dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 5. figure_05_pca_ica_projections.png: 2D PCA & FastICA projections
    #
    ----------------------------------------------------------------------
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4))
    P_tr = d_npz["P_tr"]
    S_tr = d_npz["S_tr"]
    y_tr_arr = d_npz["y_train"]
    non_ext = y_tr_arr == 0
    ext = y_tr_arr == 1

    ax1.scatter(P_tr[non_ext, 0], P_tr[non_ext, 1], c="#94A3B8", s=6,
    alpha=0.35, label="Non-Extreme Days (N=8,320)")
    ax1.scatter(P_tr[ext, 0], P_tr[ext, 1], c=CORAL, s=16, alpha=0.80,
    edgecolors="white", linewidth=0.2, label="Extreme Days ≥ 13.74 mm
    (N=439)")
    ax1.set_xlabel(f"PC1: Monsoon Moisture & Thermal Saturation ({unsup_res
    ['pca_var_explained'][0]*100:.2f}%)")
    ax1.set_ylabel(f"PC2: Radiative & Wind Advection ({unsup_res
    ['pca_var_explained'][1]*100:.2f}%)")
    ax1.set_title("2D Principal Component Projection (66.46% Combined
    Variance)", fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="upper left", frameon=True, facecolor="white",
    fontsize=8)

    ax2.scatter(S_tr[non_ext, 0], S_tr[non_ext, 1], c="#94A3B8", s=6,
    alpha=0.35, label="Non-Extreme Days (N=8,320)")
    ax2.scatter(S_tr[ext, 0], S_tr[ext, 1], c=PURPLE, s=16, alpha=0.80,
    edgecolors="white", linewidth=0.2, label="Extreme Days ≥ 13.74 mm
    (N=439)")
    ax2.set_xlabel("Independent Component 1 (IC1: Non-Gaussian Moisture
    Pulse)")
    ax2.set_ylabel("Independent Component 2 (IC2: Convective Instability
    Mode)")
```

generate_figures.py > ...

```
def generate_all_figures():
    f_indices = [split_info["feature_names"].index(f) for f in radar_feats]
    angles = np.linspace(0, 2 * np.pi, len(radar_feats), endpoint=False).
    tolist()
    angles += angles[:1]
    c_colors = [BLUE, AMBER, CORAL]
    c_names = [
        "Cluster 0: Dry Winter (0.41% Ext)",
        "Cluster 1: Pre-Monsoon (0.39% Ext)",
        "Cluster 2: Active Monsoon (94.99% Share)",
    ]
    centers = np.array(unsup_res["cluster_centers_scaled"])
    for c_i in range(3):
        vals = centers[c_i, f_indices].tolist()
        vals += vals[:1]
        ax2.plot(angles, vals, color=c_colors[c_i], lw=2.0, label=c_names
        [c_i])
        ax2.fill(angles, vals, color=c_colors[c_i], alpha=0.12)
    ax2.set_xticks(angles[:-1])
    ax2.set_xticklabels(radar_labels, fontsize=8, fontweight="bold")
    ax2.set_title("K-Means (K = 3) Standardized Centroid Profiles",
    fontweight="bold", pad=16)
    ax2.legend(loc="upper right", bbox_to_anchor=(1.28, 1.14),
    frameon=True, facecolor="white", fontsize=7.5)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_06_pca_scree_kmeans_radar.
    png"), dpi=300)
    plt.close()

    #
    ----------------------------------------------------------------------
    -
    # 7. figure_07_roc_curves.png: Classifier ROC curves with random
    baseline
    #
    ----------------------------------------------------------------------
    -
    y_te = d_npz["y_test"]
    model_specs = [
        ("Logistic Regression", d_npz["pr_lr"], d_npz["yp_lr"], CORAL, "-"),
        ("Random Forest", d_npz["pr_rf"], d_npz["yp_rf"], NAVY, "-"),
        ("Support Vector Machine (RBF)", d_npz["pr_svm"], d_npz["yp_svm"],
        TEAL, "--"),
        ("Decision Tree (CART)", d_npz["pr_dt"], d_npz["yp_dt"], AMBER, "-.
        "),
        ("k-Nearest Neighbours (k=5)", d_npz["pr_knn"], d_npz["yp_knn"],
        PURPLE, ":"),
    ]
```

generate_figures.py > ...

```
def generate_all_figures():
    fig, ax = plt.subplots(figsize=(7.2, 4.8))
    for name, pr, _, col, ls in model_specs:
        fpr, tpr, _ = roc_curve(y_te, pr)
        auc_v = roc_auc_score(y_te, pr)
        ax.plot(fpr, tpr, color=col, linestyle=ls, lw=2.2, label=f"{name}
        (ROC-AUC = {auc_v:.4f})")
    ax.plot([0, 1], [0, 1], color=SLATE, linestyle="--", lw=1.2,
    label="Random Classifier Baseline (AUC = 0.5000)")
    ax.set_xlabel("False Positive Rate (1 − Specificity)")
    ax.set_ylabel("True Positive Rate (Recall / Probability of Detection)")
    ax.set_title("Out-of-Sample Receiver Operating Characteristic Curves
    (2019–2024)", fontweight="bold")
    ax.grid(True, linestyle=":", alpha=0.6)
    ax.legend(loc="lower right", frameon=True, facecolor="white",
    fontsize=8.3)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_07_roc_curves.png"), dpi=300)
    plt.close()

    #
    ------------------------------------------------------------------------
    -
    # 8. figure_08_pr_curves.png: Precision-Recall curves with 7.48%
    baseline
    #
    ------------------------------------------------------------------------
    -
    fig, ax = plt.subplots(figsize=(7.2, 4.8))
    for name, pr, _, col, ls in model_specs:
        prec, rec, _ = precision_recall_curve(y_te, pr)
        ap_v = average_precision_score(y_te, pr)
        ax.plot(rec, prec, color=col, linestyle=ls, lw=2.2, label=f"{name}
        (PR-AUC = {ap_v:.4f})")
    base_rate = float(y_te.mean())
    ax.axhline(base_rate, color=SLATE, linestyle="--", lw=1.4,
    label=f"Climatological Prevalence Baseline ({base_rate*100:.2f}%)")
    ax.set_xlabel("Recall (Probability of Detection)")
    ax.set_ylabel("Precision (Positive Predictive Value)")
    ax.set_ylim(0, 1.02)
    ax.set_title("Out-of-Sample Precision–Recall Curves on Holdout Set
    (2019–2024)", fontweight="bold")
    ax.grid(True, linestyle=":", alpha=0.6)
    ax.legend(loc="upper right", frameon=True, facecolor="white",
    fontsize=8.3)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_08_pr_curves.png"), dpi=300)
```

generate_figures.py > ...

```
def generate_all_figures():
    -
    # 10. figure_10_feature_attribution.png: Multi-panel feature attribution
    #
    ------------------------------------------------------------------------
    -
    feat_df = pd.read_csv(os.path.join(res_dir, "feature_importances.csv")).
    head(10).iloc[::-1]
    fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(13.2, 4.4),
    sharey=True)
    y_pos = np.arange(len(feat_df))

    ax1.barh(y_pos, feat_df["RF_Gini_Importance"] * 100, height=0.38,
    label="Random Forest Gini (%)", color=NAVY)
    ax1.barh(y_pos - 0.38, feat_df["DT_Gini_Importance"] * 100, height=0.
    38, label="Decision Tree Gini (%)", color=AMBER, alpha=0.85)
    ax1.set_yticks(y_pos)
    ax1.set_yticklabels(feat_df["Feature"], fontweight="bold")
    ax1.set_xlabel("Impurity Importance (%)")
    ax1.set_title("Tree Gini Importance (RF vs. CART)", fontweight="bold")
    ax1.grid(True, axis="x", linestyle=":", alpha=0.6)
    ax1.legend(loc="lower right", frameon=True, facecolor="white",
    fontsize=7.8)

    bar_cols = [CORAL if v > 0 else BLUE for v in feat_df["LR_Coefficient"]]
    ax2.barh(y_pos, feat_df["LR_Coefficient"], height=0.55, color=bar_cols,
    alpha=0.88)
    ax2.axvline(0, color=SLATE, lw=1.0)
    ax2.set_xlabel("Standardized Log-Odds Weight (β)")
    ax2.set_title("Logistic Regression Standardized Coefficients",
    fontweight="bold")
    ax2.grid(True, axis="x", linestyle=":", alpha=0.6)

    ax3.barh(y_pos, feat_df["RF_Permutation_AUC_Drop"], height=0.55,
    color=TEAL, alpha=0.88)
    ax3.set_xlabel("Holdout Permutation ΔROC-AUC")
    ax3.set_title("Test-Set Permutation Importance (ΔAUC)",
    fontweight="bold")
    ax3.grid(True, axis="x", linestyle=":", alpha=0.6)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_10_feature_attribution.png"),
    dpi=300)
    plt.close()

    #
    ------------------------------------------------------------------------
    -
```

generate_figures.py > ...

```
def generate_all_figures():
    # 9. figure_09_confusion_matrices.png: 5-model grid with counts & row %
    #
    ------------------------------------------------------------------------
    -
    perf_map = {r["Model"]: r for r in verified["test_performance"]}
    fig, axes = plt.subplots(1, 5, figsize=(14.5, 3.3))
    ordered_models = [
        ("Logistic Regression", "Logistic Regression", d_npz["yp_lr"]),
        ("Decision Tree", "Decision Tree (CART)", d_npz["yp_dt"]),
        ("Support Vector Machine", "RBF-Kernel SVM", d_npz["yp_svm"]),
        ("Random Forest", "Random Forest", d_npz["yp_rf"]),
        ("k-Nearest Neighbours", "k-Nearest Neighbours", d_npz["yp_knn"]),
    ]
    for idx, (m_key, m_disp, yp) in enumerate(ordered_models):
        ax = axes[idx]
        pm = perf_map[m_key]
        cm = confusion_matrix(y_te, yp)
        cm_pct = cm.astype(float) / cm.sum(axis=1, keepdims=True) * 100.0
        ax.imshow(cm_pct, cmap="Blues", vmin=0, vmax=100)
        title_str = f"{m_disp}\n(Rec: {pm['Recall']*100:.2f}%, F1: {pm
        ['F1_Score']:.4f})"
        ax.set_title(title_str, fontsize=8.6, fontweight="bold")
        ax.set_xticks([0, 1])
        ax.set_yticks([0, 1])
        ax.set_xticklabels(["Pred 0", "Pred 1"], fontsize=8)
        ax.set_yticklabels(["Actual 0\n(2,028)", "Actual 1\n(164)"] if idx
        == 0 else ["", ""], fontsize=8)
        for r in range(2):
            for c in range(2):
                val_cnt = cm[r, c]
                val_p = cm_pct[r, c]
                lbl = [["TN", "FP"], ["FN", "TP"]][r][c]
                txt_col = "white" if val_p > 55 else NAVY
                ax.text(
                    c, r, f"{lbl}: {val_cnt}\n({val_p:.2f}%)",
                    ha="center", va="center", color=txt_col, fontsize=8.2,
                    fontweight="bold",
                )
    plt.suptitle(
        "Holdout Confusion Matrices (2019–2024, N = 2,192 Days): Raw Counts
        and Row-Normalized Percentages",
        fontsize=10.5, fontweight="bold", y=1.02,
    )
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_09_confusion_matrices.png"),
    dpi=300, bbox_inches="tight")
```

generate_figures.py > ...

```
def generate_all_figures():
    # 11. figure_11_calibration_reliability.png: Reliability curves & Brier
    #
    ------------------------------------------------------------------------
    -
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.4, 4.3), gridspec_kw=
    {"width_ratios": [1.25, 1.0]})
    ax1.plot([0, 1], [0, 1], "k--", lw=1.3, label="Perfect Calibration
    (Diagonal)")
    for name, pr, _, col, ls in model_specs:
        bins_cal = np.linspace(0, 1, 8)
        bin_ids = np.digitize(pr, bins_cal) - 1
        mean_p, frac_pos = [], []
        for b in range(len(bins_cal) - 1):
            m = bin_ids == b
            if m.sum() >= 8:
                mean_p.append(pr[m].mean())
                frac_pos.append(y_te[m].mean())
        brier_v = brier_score_loss(y_te, pr)
        ax1.plot(mean_p, frac_pos, marker="o", color=col, linestyle=ls,
        lw=2.0, label=f"{name} (Brier = {brier_v:.4f})")
    ax1.set_xlabel("Mean Predicted Extreme Rainfall Probability")
    ax1.set_ylabel("Observed Empirical Extreme Frequency")
    ax1.set_title("Probability Calibration Reliability Diagram (2019–2024)
    ", fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="upper left", frameon=True, facecolor="white",
    fontsize=7.8)

    names_b = ["RBF-SVM\n(Platt)", "k-NN\n(k=5)", "Random Forest\n(Bagged)
    ", "Decision Tree\n(CART)", "Logistic Reg.\n(Balanced)"]
    briers = [
        brier_score_loss(y_te, d_npz["pr_svm"]),
        brier_score_loss(y_te, d_npz["pr_knn"]),
        brier_score_loss(y_te, d_npz["pr_rf"]),
        brier_score_loss(y_te, d_npz["pr_dt"]),
        brier_score_loss(y_te, d_npz["pr_lr"]),
    ]
    b_cols = [TEAL, PURPLE, NAVY, AMBER, CORAL]
    bars = ax2.bar(names_b, briers, color=b_cols, alpha=0.88, width=0.52)
    for bar, v in zip(bars, briers):
        ax2.text(bar.get_x() + bar.get_width() / 2, v + 0.004, f"{v:.4f}",
        ha="center", va="bottom", fontsize=8.5, fontweight="bold")
    ax2.set_ylabel("Brier Calibration Loss (Lower is Better)")
    ax2.set_ylim(0, 0.195)
    ax2.set_title("Holdout Brier Score Comparison Across Classifiers",
    fontweight="bold")
```

generate_figures.py > ...

```python
def generate_all_figures():
    # 12. figure_12_threshold_cost_sweep.png: RF trade-off & 15:1 cost curve
    #
    # ------------------------------------------------------------------------
    sim_df = pd.read_csv(os.path.join(res_dir,
    "operational_threshold_simulation.csv"))
    rf_sub = sim_df[sim_df["Model"] == "Random Forest"].sort_values
    ("Decision_Threshold")
    lr_sub = sim_df[sim_df["Model"] == "Logistic Regression"].sort_values
    ("Decision_Threshold")

    t_sweep = {r["Threshold"]: r for r in verified["threshold_cost_sweep"]
    ["test_sweep"]}
    tau_cv = verified["threshold_cost_sweep"]["tau_cv_star"]
    tau_ph = verified["threshold_cost_sweep"]["tau_posthoc_star"]
    r_cv = t_sweep[tau_cv]
    r_50 = t_sweep[0.50]

    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11.6, 4.4))
    ax1.plot(rf_sub["Decision_Threshold"], rf_sub["Recall"] * 100,
    color=TEAL, lw=2.2, marker="o", markersize=4, label="Recall / POD (%)")
    ax1.plot(rf_sub["Decision_Threshold"], rf_sub["Precision"] * 100,
    color=BLUE, lw=2.2, marker="s", markersize=4, label="Precision / PPV (%)")
    ax1.plot(rf_sub["Decision_Threshold"], rf_sub["F1_Score"] * 100,
    color=NAVY, lw=2.2, marker="^", markersize=4, label="F1-Score (%)")
    ax1.axvline(tau_cv, color=CORAL, linestyle="--", lw=1.9, label=f"OOF &
    Test Min τ* = {tau_cv:.2f} (Rec={r_cv['Recall']*100:.2f}%)")
    ax1.axvline(0.50, color=SLATE, linestyle=":", lw=1.6, label=f"Default
    Cutoff τ = 0.50 (Rec={r_50['Recall']*100:.2f}%)")
    ax1.set_xlabel("Decision Probability Threshold (τ)")
    ax1.set_ylabel("Performance Metric (%)")
    ax1.set_title("Random Forest Precision, Recall, and F1 Trade-Off Curve",
    fontweight="bold")
    ax1.grid(True, linestyle=":", alpha=0.6)
    ax1.legend(loc="center left", frameon=True, facecolor="white", fontsize=7.
    6)

    ax2.plot(rf_sub["Decision_Threshold"], rf_sub["Total_Cost_15x"],
    color=NAVY, lw=2.4, marker="o", markersize=4, label="Random Forest
    Holdout 15:1 Loss")
    ax2.plot(lr_sub["Decision_Threshold"], lr_sub["Total_Cost_15x"],
    color=CORAL, lw=2.0, linestyle="--", label="Logistic Regression Holdout
    15:1 Loss")
    ax2.axvline(tau_cv, color=PURPLE, linestyle="-.", lw=1.9,
    label=f"Leakage-Free OOF τ_CV* = {tau_cv:.2f} (Loss = {r_cv
    ['Total_Cost_15x']})")
    ax2.axvline(0.50, color=SLATE, linestyle=":", lw=1.6, label=f"Default
    Cutoff τ = 0.50 (Loss = {r_50['Total_Cost_15x']})")
```

generate_figures.py > ...

```python
def generate_all_figures():
    Cutoff τ = 0.50 (Loss = {r_50['Total_Cost_15x']})")
    ax2.scatter([tau_cv, 0.50], [r_cv["Total_Cost_15x"], r_50
    ["Total_Cost_15x"]], color=[PURPLE, SLATE], s=85, zorder=5)
    ax2.set_xlabel("Decision Probability Threshold (τ)")
    ax2.set_ylabel("Asymmetric Cost Loss L(τ) = 15·FN(τ) + 1·FP(τ)")
    ax2.set_title("Asymmetric 15:1 Disaster Cost Curve (OOF τ_CV* vs. Holdout
    Sweep)", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper left", frameon=True, facecolor="white", fontsize=7.
    8)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_12_threshold_cost_sweep.png"),
    dpi=300)
    plt.close()

    #
    # ------------------------------------------------------------------------
    # 13. figure_13_sept2024_case_study.png: 20-30 Sept 2024 Kathmandu flood
    #
    # ------------------------------------------------------------------------
    sep_records = verified["september_2024_case_study"]
    d_labels = [pd.to_datetime(r["date"]).strftime("%d Sep") for r in
    sep_records]
    x_s = np.arange(len(d_labels))
    rain_vals = np.array([r["PRECTOTCORR"] for r in sep_records])
    rf_s24_probs = np.array([r["rf_prob_task_a"] for r in sep_records])
    lr_s24_probs = np.array([r["lr_prob_task_a"] for r in sep_records])
    lr24_s24_probs = np.array([r["lr_prob_task_b"] for r in sep_records])

    fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10.5, 5.2), sharex=True,
    gridspec_kw={"height_ratios": [1.05, 1.1]})
    b_cols = [CORAL if r >= 13.74 else BLUE for r in rain_vals]
    bars = ax1.bar(x_s, rain_vals, color=b_cols, width=0.55, alpha=0.90,
    label="Daily MERRA-2 Rainfall (mm/day)")
    ax1.axhline(13.74, color=CORAL, linestyle="--", lw=1.6, label="All-Day
    Extreme Threshold P95 = 13.74 mm/day")
    ax1.axhline(22.752, color=AMBER, linestyle="-.", lw=1.6, label="Wet-Day
    Threshold R95p = 22.75 mm/day")
    for bar, v in zip(bars, rain_vals):
        ax1.text(bar.get_x() + bar.get_width() / 2, v + 1.8, f"{v:.2f}",
        ha="center", va="bottom", fontsize=7.8, fontweight="bold")
    ax1.set_ylim(0, 108)
    ax1.set_ylabel("Rainfall (mm/day)", fontweight="bold")
    ax1.set_title("Holdout Case Study: 20–30 September 2024 Kathmandu Extreme
    Deluge & Classifier Warnings", fontweight="bold")
    ax1.grid(True, axis="y", linestyle=":", alpha=0.6)
    ax1.legend(loc="upper left", frameon=True, facecolor="white", fontsize=8)
```

generate_figures.py > ...

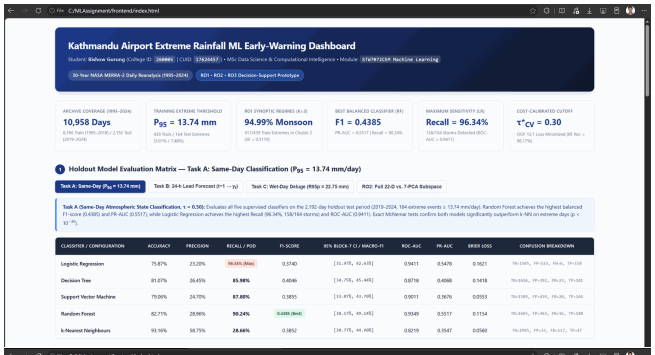
```python
def generate_all_figures():
    ax2.plot(x_s, lr_s24_probs, marker="s", lw=2.3, linestyle="-",
    color=CORAL, label="Logistic Regression (Task A Same-Day)")
    ax2.plot(x_s, rf_s24_probs, marker="o", lw=2.3, linestyle="-",
    color=NAVY, label="Random Forest (Task A Same-Day)")
    ax2.plot(x_s, lr24_s24_probs, marker="^", lw=1.9, linestyle="--",
    color=TEAL, label="Logistic Regression (Task B 24-h Lead Time)")
    ax2.axhline(tau_cv, color=AMBER, linestyle=":", lw=1.6,
    label=f"Cost-Calibrated OOF Cutoff (τ_CV* = {tau_cv:.2f})")
    ax2.axhline(0.50, color=SLATE, linestyle="--", lw=1.4, label="Default
    Cutoff (τ = 0.50)")
    ax2.axhline(0.70, color=PURPLE, linestyle="-.", lw=1.4,
    label="High-Priority Escalation (τ = 0.70)")
    ax2.axvspan(3.6, 8.4, color=CORAL, alpha=0.10, label="Active 5-Day
    Extreme Deluge Window (24–28 Sep)")
    ax2.set_xticks(x_s)
    ax2.set_xticklabels(d_labels, fontweight="bold")
    ax2.set_ylim(0, 1.08)
    ax2.set_xlabel("Date (20–30 September 2024 Holdout Period)")
    ax2.set_ylabel("Predicted Extreme Probability", fontweight="bold")
    ax2.grid(True, linestyle=":", alpha=0.6)
    ax2.legend(loc="upper left", ncol=2, frameon=True, facecolor="white",
    fontsize=7.5)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_13_sept2024_case_study.png"),
    dpi=300)
    plt.close()

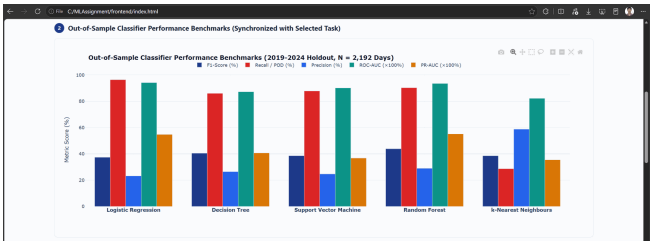
    #
    # ------------------------------------------------------------------------
    # 14. figure_14_lead_time_wetday_sensitivity.png: Tasks A, B, and C
    #
    # ------------------------------------------------------------------------
    sens = verified["sensitivity_tasks"]
    tasks_labels = [
        "Task A: Same-Day\nRandom Forest",
        "Task A: Same-Day\nLogistic Reg.",
        "Task B: 24-h Lead\nLogistic Reg.",
        "Task B: 24-h Lead\nk-NN (k=5)",
        "Task C: Wet R95p\nLogistic Reg.",
    ]
    t_keys = [
        "Task_A_SameDay_RF",
        "Task_A_SameDay_LR",
        "Task_B_24hLeadTime_LR",
        "Task_B_24hLeadTime_kNN",
        "Task_C_WetDayR95p_LR",
    ]
```

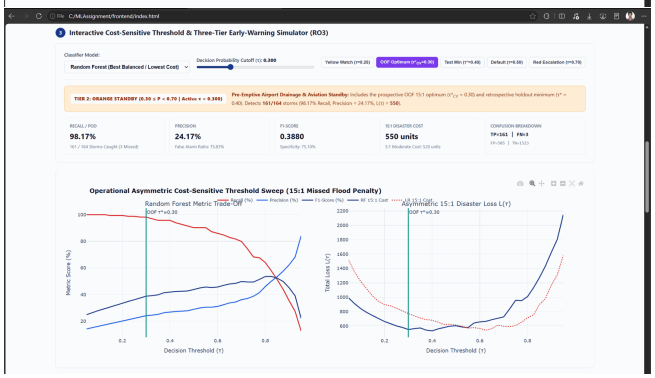
```python
    rec_vals = [sens[k]["Recall"] * 100 for k in t_keys]
    roc_vals = [sens[k]["ROC_AUC"] * 100 for k in t_keys]
    pr_vals = [sens[k]["PR_AUC"] * 100 for k in t_keys]
    f1_vals = [sens[k]["F1_Score"] * 100 for k in t_keys]

    fig, ax = plt.subplots(figsize=(10.8, 4.4))
    x_t = np.arange(len(tasks_labels))
    w_b = 0.20
    ax.bar(x_t - 1.5 * w_b, rec_vals, width=w_b, color=CORAL, label="Recall /
    POD (%)")
    ax.bar(x_t - 0.5 * w_b, roc_vals, width=w_b, color=NAVY, label="ROC-AUC
    (×100%)")
    ax.bar(x_t + 0.5 * w_b, pr_vals, width=w_b, color=TEAL, label="PR-AUC
    (×100%)")
    ax.bar(x_t + 1.5 * w_b, f1_vals, width=w_b, color=AMBER, label="F1-Score
    (×100%)")
    for i in range(len(x_t)):
        ax.text(x_t[i] - 1.5 * w_b, rec_vals[i] + 1.2, f"{rec_vals[i]:.1f}%",
        ha="center", fontsize=7.5, fontweight="bold")
        ax.text(x_t[i] - 0.5 * w_b, roc_vals[i] + 1.2, f"{roc_vals[i]:.1f}%",
        ha="center", fontsize=7.5, fontweight="bold")
        ax.text(x_t[i] + 0.5 * w_b, pr_vals[i] + 1.2, f"{pr_vals[i]:.1f}%",
        ha="center", fontsize=7.5, fontweight="bold")
        ax.text(x_t[i] + 1.5 * w_b, f1_vals[i] + 1.2, f"{f1_vals[i]:.1f}%",
        ha="center", fontsize=7.5, fontweight="bold")
    ax.set_xticks(x_t)
    ax.set_xticklabels(tasks_labels, fontweight="bold")
    ax.set_ylim(0, 108)
    ax.set_ylabel("Metric Score (%)")
    ax.set_title("Sensitivity Comparison: Task A (Same-Day) vs. Task B (24-h
    Lead Time) vs. Task C (Wet-Day R95p)", fontweight="bold")
    ax.grid(True, axis="y", linestyle=":", alpha=0.6)
    ax.legend(loc="upper right", ncol=2, frameon=True, facecolor="white",
    fontsize=8.5)
    plt.tight_layout()
    plt.savefig(os.path.join(fig_dir, "figure_14_lead_time_wetday_sensitivity.
    png"), dpi=300)
    plt.close()
```

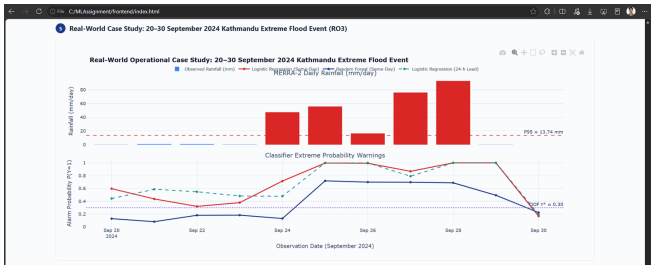
D. OPERATIONAL DASHBOARD PROTOTYPE

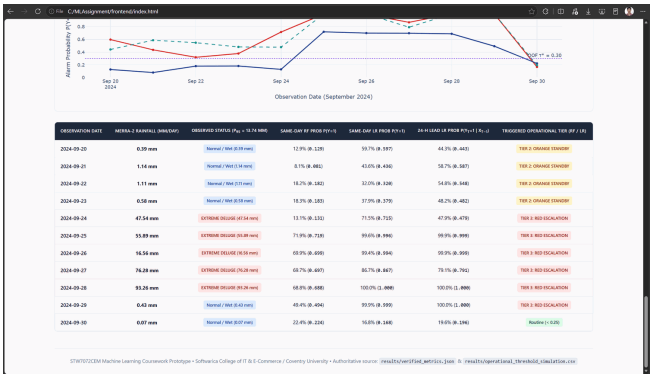









C:/MLAssignment/frontend/index.html
Real-World Case Study: 20–30 September 2024 Kathmandu Extreme Flood Event (RO3)
Real-World Operational Case Study: 20–30 September 2024 Kathmandu Extreme Flood Event
Observed Rainfall (mm)
Logistic Regression (24-h Lead)
NEBSA-2 Daily Rainfall (mm/day)
Rainfall (mm/day)
P95 = 13.74 mm
Classifier Extreme Probability Warnings
Alarm Probability P(Y=1)
0.07 τ* = 0.30
Sep 20 2024
Sep 22
Sep 24
Sep 26
Sep 28
Sep 30
Observation Date (September 2024)


C:/MLAssignment/frontend/index.html
Alarm Probability P(Y=1)
0.07 τ* = 0.30
Sep 20 2024
Sep 22
Sep 24
Sep 26
Sep 28
Sep 30
Observation Date (September 2024)
OBSERVATION DATE
NEBSA-2 RAINFALL (MM/DAY)
OBSERVED STATUS (P_{95} = 13.74 MM)
SAME-DAY RF PROB P(Y=1)
SAME-DAY LR PROB P(Y=1)
24-H LEAD LR PROB $P(Y_{t+1} \mid X_{t:t})$
TRIGGERED OPERATIONAL TIER (RF / LR)
2024-09-20 0.39 mm Normal / Wet (0.39 mm) 12.9% (0.129) 59.7% (0.597) 44.3% (0.443) TIER 2: ORANGE STANDBY
2024-09-21 1.14 mm Normal / Wet (1.14 mm) 8.1% (0.081) 43.6% (0.436) 58.7% (0.587) TIER 2: ORANGE STANDBY
2024-09-22 1.11 mm Normal / Wet (1.11 mm) 18.2% (0.182) 32.0% (0.320) 54.8% (0.548) TIER 2: ORANGE STANDBY
2024-09-23 0.58 mm Normal / Wet (0.58 mm) 18.3% (0.183) 37.9% (0.379) 48.2% (0.482) TIER 2: ORANGE STANDBY
2024-09-24 47.54 mm EXTREME DELUGE (47.54 mm) 13.1% (0.131) 71.5% (0.715) 47.9% (0.479) TIER 2: RED ESCALATION
2024-09-25 55.89 mm EXTREME DELUGE (55.89 mm) 71.9% (0.719) 99.6% (0.996) 99.9% (0.999) TIER 3: RED ESCALATION
2024-09-26 16.56 mm EXTREME DELUGE (16.56 mm) 69.9% (0.699) 99.4% (0.994) 99.9% (0.999) TIER 3: RED ESCALATION
2024-09-27 76.28 mm EXTREME DELUGE (76.28 mm) 69.7% (0.697) 86.7% (0.867) 79.1% (0.791) TIER 3: RED ESCALATION
2024-09-28 93.26 mm EXTREME DELUGE (93.26 mm) 68.8% (0.688) 100.0% (1.000) 100.0% (1.000) TIER 3: RED ESCALATION
2024-09-29 0.43 mm Normal / Wet (0.43 mm) 40.4% (0.404) 99.9% (0.999) 100.0% (1.000) TIER 3: RED ESCALATION
2024-09-30 0.07 mm Normal / Wet (0.07 mm) 22.4% (0.224) 16.8% (0.168) 19.6% (0.196) Routine (< 0.25)
STW7072CEM Machine Learning Coursework Prototype • Softwarica College of IT & E-Commerce / Coventry University • Authoritative source: results/verified_metrics.json & results/operational_threshold_simulation.csv